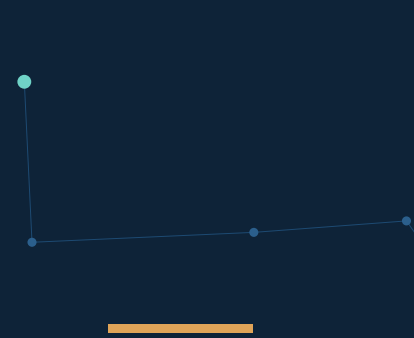


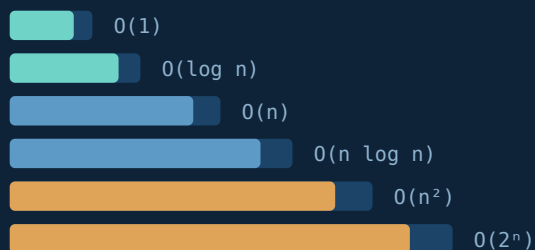


THE PLAIN-ENGLISH ANSWER BOOK

Say It Simply



627 AI/ML interview questions,
answered in words you can
actually say out loud.



Prepared for Ayush Poojari

First edition · 2026



Say It Simply

627 AI/ML interview questions,
answered in words you can
actually say out loud.

Every answer here is written the way you would say it to a person, not the way a textbook would write it. Short sentences, ordinary words, and nothing you cannot defend in a follow-up question.

Prepared for Ayush Poojari

First edition · 2026

How to Use This Book

Your question bank has 627 questions with short technical answers. This book has the same 627 questions with the answers rewritten in plain English — the way you would actually say them to a person sitting across a table.

Nothing has been dumbed down. The facts are identical. What has changed is the wording: ordinary words instead of jargon, short sentences instead of dense ones, and a plain explanation of any term that cannot be avoided.

Why the wording matters

An interview is a conversation, not a written exam. When you recite a textbook sentence, three things go wrong. You sound like you memorised it. You use a term you cannot then explain when they ask. And you talk for too long, which walks you into territory you cannot defend.

A plain answer avoids all three. It also proves something a technical answer cannot: that you actually understand the idea, because explaining something simply is only possible once you do.

KEY INSIGHT | The rule that runs through this whole book

Answer in two or three sentences, then stop.

Silence after a short answer is not awkward — it is an invitation for them to ask the follow-up they care about. Long answers are how candidates talk themselves into a corner. If they want more, they will ask, and then you give them more.

How each answer is written

Every answer here follows the same shape:

The plain sentence. What the thing is, in words a non-specialist would follow.

Why it matters, or where it bites. One line of consequence, because that is what separates someone who has used it from someone who has read about it.

Your hook, where there is one. Many answers end with a link to your own projects. Use it. An answer that lands on something you built is worth far more than a correct definition.

NOTE | What to do when you do not know

Say so. “I don’t know” followed by “here’s how I’d find out” is a good answer. Every experienced interviewer prefers it to confident invention, and invention is the one failure they will not forgive.

The order to read it in

Read **Chapter 2** first, then **Chapter 17**. Those two attack your own resume and your own projects, and they are the questions you are most likely to be asked and least likely to have rehearsed.

Everything else you can read in any order. The chapter each section came from in your original bank is marked, so you can cross-reference the two.

WATCH OUT | One thing before anything else

Fix the two stale README files, and check that your CI pipeline triggers on the branch you actually use. That is an hour of work and it is worth more than any answer in this book, because an interviewer who reads a stale README concludes your resume is inflated when it is actually accurate.

Contents at a Glance

How to Use This Book	iii
Part I — The Conversation	1
1. The Opening Questions	2
2. Defending Your Resume	6
Part II — The Programming Rounds	11
3. Python	12
4. SQL, C++ and MongoDB	16
Part III — Data and Classical ML	22
5. Statistics	23
6. Pandas, NumPy and Looking at Data	27
7. Classical Machine Learning	31
8. Imbalanced Data and Metrics	37
Part IV — Deep Learning and Vision	42
9. Deep Learning	43
10. CNNs, YOLO and OpenCV	49
Part V — Language Models	57
11. NLP and Transformers	58
12. Embeddings, Vector Search and RAG	65
13. Prompting, LangChain and Agents	72
Part VI — Shipping It	77
14. MLOps: MLflow, Docker and CI/CD	78

15. APIs, Streamlit and Git	84
16. ML System Design	89
Part VII — Your Work and the Last Mile	92
17. Your Four Projects	93
18. The Coding Round	101
19. What to Ask Them, and the Final Checklist	105

Contents

How to Use This Book	iii
-----------------------------	------------

Part I — The Conversation	1
----------------------------------	----------

1. The Opening Questions	2
Who you are	2
Practical questions	4
2. Defending Your Resume	6
The churn accuracy question	6
The business percentages	7
The detection scores	7
The honest gaps	8
The uncomfortable ones	8

Part II — The Programming Rounds	11
---	-----------

3. Python	12
Data types	12
Functions and structure	13
Errors, classes and files	14
4. SQL, C++ and MongoDB	16
SQL: the basics	16
SQL: the parts that get asked live	17
C++	18
MongoDB	20

Part III — Data and Classical ML

22

5. Statistics 23

Describing data 23

Testing and uncertainty 24

The ways data misleads 25

6. Pandas, NumPy and Looking at Data 27

The basics 27

NumPy 28

Looking at the data 29

7. Classical Machine Learning 31

The core ideas 31

The algorithms 32

8. Imbalanced Data and Metrics 37

Imbalanced data 37

The metrics 39

Part IV — Deep Learning and Vision

42

9. Deep Learning 43

What a network is 43

PyTorch in practice 46

10. CNNs, YOLO and OpenCV 49

Convolutional networks 49

YOLO 51

OpenCV 55

Part V — Language Models **57**

11. NLP and Transformers **58**

Classic NLP 58

Transformers 60

12. Embeddings, Vector Search and RAG **65**

Embeddings and vector search 65

RAG 68

13. Prompting, LangChain and Agents **72**

Prompting 72

LangChain and LangGraph 73

Agents 75

Part VI — Shipping It **77**

14. MLOps: MLflow, Docker and CI/CD **78**

The idea 78

Docker 79

CI/CD 80

Monitoring 81

15. APIs, Streamlit and Git **84**

REST and FastAPI 84

Git 86

16. ML System Design **89**

Part VII — Your Work and the Last Mile **92**

17. Your Four Projects	93
SafeSight	93
DocuMind	95
Fire detection	97
Churn pipeline	98
18. The Coding Round	101
Complexity	101
19. What to Ask Them, and the Final Checklist	105
Questions to ask them	105
Before you walk in	106

PART I

The Conversation

The questions that are not about technology at all, and the ones that attack the numbers on your own resume. These decide more interviews than the technical rounds do.

-
- 01 The Opening Questions
 - 02 Defending Your Resume

The Opening Questions

The behavioural round, in words you can actually say

FOUNDATION

PREREQS: none

These are not technical questions and they are not small talk either. They are testing three things: can you hold a conversation, do you own your work, and will you be difficult to sit next to. Keep every answer under ninety seconds, and let them ask the follow-up.

WHAT YOU WILL LEARN

- Open with a 60-second summary that invites the right follow-up
- Tell a failure story that ends in a fix, not an excuse
- Name a real weakness and the correction you made
- Close the interview by asking something that shows interest

Who you are

Q1.1 — Tell me about yourself.

Say this: “I’m an AI/ML engineer, and I’m doing my M.Sc. in Data Science and AI at BITS Pilani. My last role was an internship at Infosys, where I built SafeSight — a workplace safety system. A camera model spots workers who aren’t wearing the right safety gear, and a second part looks up which safety rule they’ve broken and shows it to the supervisor. Outside that I’ve built a document question-answering tool, a fire detector, and a customer-churn pipeline that’s deployed and running. I’m looking for a job where I own a model from training all the way to production.”

That’s about forty seconds. It gives them four things to ask about, and the one they’ll pick is SafeSight — which is what you want.

Q1.2 — Why AI/ML?

Don’t be romantic about it. Say: “Because you can tell whether you were right. You ship a model, the number tells you if it worked, and you can’t argue with it. Computer vision got me first — you can literally watch the model succeed or fail frame by frame.”

Q1.3 — Walk me through your resume.

Go in order and keep it to two minutes. Bachelor's in IT at Thakur, then the AI/ML programme at IIT Hyderabad, then the M.Sc. at BITS, then the Infosys internship. End on the internship — that's where you want them to dig.

Q1.4 — What is your greatest strength?

Pick one and prove it with a fact. "I finish things. Every project I've built has a Dockerfile and a running endpoint, not just a notebook that worked once on my laptop."

Q1.5 — What is your greatest weakness?

Name a real one and say what you did about it. "I used to spend all my time on the model and treat deployment as something I'd sort out later. On the churn project I made myself start with the Dockerfile and the pipeline instead. It's still the part I have to be deliberate about."

Never say "I'm a perfectionist." Everyone has heard it and nobody believes it.

Q1.6 — Why should we hire you?

"I've already built the kind of system you're hiring for — a detection model, a retrieval layer, an API, a dashboard, and a container to run it in. I'll need to learn your domain, but not your stack."

Q1.7 — Tell me about a time you failed.

Use a real one with a fix attached. The PPE detector is good: the small items like goggles were being missed. Say: "At first I assumed the model was too small. When I actually looked at the data, there were far fewer goggle examples than helmet examples — it was a data problem, not a model problem. I rebalanced the dataset and it improved."

That story is strong because it shows you checked instead of guessing.

Q1.8 — Tell me about a conflict with a teammate.

Keep it about the work, never about the person. "We disagreed on which approach to take. Rather than argue it out, I suggested we measure both and let the numbers decide. They did."

Never tell a story where the other person is just an idiot. The interviewer will assume you'll say the same about them.

Q1.9 — How do you handle criticism of your work?

"In a code review I'd much rather be corrected than ship something broken." Then give one real example of feedback you acted on.

Q1.10 — Describe a time you worked under a tight deadline.

Use the internship. Talk about what you **cut**, not how hard you worked. Deciding what not to build is the skill they're testing. "I dropped the night-time footage work and shipped the daytime version first, because that covered most of the shift patterns."

Q1.11 — How do you work in a team?

Be concrete. Don't say "I'm a good communicator" — everybody says that. Say: "I keep work on its own branch and open a pull request early, while it's still small enough for someone to actually read."

Q1.12 — Where do you see yourself in five years?

"Owning a machine learning system in production end to end, and helping juniors with the deployment side — because that's the part most people pick up last."

Q1.13 — Why are you leaving Infosys?

Stay neutral and look forward. "It was a fixed-term internship. I'm looking for a role where I own a system for longer than three months."

Never criticise them. It tells the interviewer what you'll say about this company later.

Q1.14 — What do you know about our company?

This is homework you cannot skip. Name their product, one recent launch, and one guess about where machine learning sits in it. Then turn it into a question: "I'd guess the recommendation side is where most of your ML effort goes — is that right?"

Turning it into a question is the trick. It changes an exam into a conversation.

Practical questions

Q1.15 — What are your salary expectations?

Give a range you've actually researched for Mumbai at your level, and say you're flexible. "Based on what I've seen for AI/ML roles here at my level, I'm looking at X to Y. Honestly I care more about how much I'd own than the exact number."

Never say "whatever you offer." It reads as having no idea what you're worth.

Q1.16 — How do you keep up with the field?

Name real things or skip the question. Papers you've actually read, the Ultralytics release notes, the LangGraph changelog. "I follow AI news" means nothing and they know it.

Q1.17 — Are you comfortable relocating?

Your resume says you're open to it, so say yes plainly. Contradicting your own resume is an instant credibility problem.

Q1.18 — You're still doing your M.Sc. How will you manage both?

Have a concrete plan ready. "The course is structured so the work sits outside office hours, and I built all four of my projects while studying — so it's not theoretical."

Q1.19 — What's your biggest technical achievement?

Pick one and go deep. SafeSight is the strongest: it's the only one with real users, a real deployment, and a measured number attached to it.

Q1.20 — Do you prefer research or engineering?

For most hiring managers, the honest engineering answer wins. “Engineering. I like reading the paper, but what I actually enjoy is getting it to run on a live video feed without falling over.”

Q1.21 — What would you do differently on your last project?

This is testing self-awareness, so give a real answer. “I’d have built the evaluation set before training anything, so every run was comparable from day one. I ended up with numbers I couldn’t properly compare.”

Q1.22 — Do you have any questions for us?

Always yes. Chapter 19 has the list. Turning up with no questions reads as no interest, and it’s the easiest thing in the whole interview to get right.

PRACTICE | Before the interview

Write out your answer to Q1.1 and say it out loud, timed. If it runs past sixty seconds, cut it. Then do the same for Q1.7 and Q1.5 — the failure and the weakness — because those are the two people improvise badly and regret.

CHAPTER RECAP

- Lead with a forty-second summary that gives them four things to ask about.
- Every story needs a fix attached, not just a problem.
- Talk about what you cut under deadline, not how hard you worked.
- Never criticise a past employer or a teammate.
- Research the company. One specific fact turns the question into a conversation.
- Always have questions for them.

Defending Your Resume

The questions that attack your own numbers

FOUNDATION

PREREQS: none

These go straight at the specific figures you wrote down. A good interviewer will find every one of them, and most candidates have never rehearsed a single one. Read this chapter before anything else in the book — it is the one that decides interviews.

WHAT YOU WILL LEARN

- Raise your weakest number yourself, before they do
- Know exactly where every percentage on your resume came from
- Explain what your detection scores actually mean
- Say “I don’t have a number for that” without it hurting you

The churn accuracy question

Q2.1 — Your churn model gets 76% accuracy. What’s the base rate?

This is the dangerous one. In that dataset, 5,174 out of 7,043 customers did not churn. So a model that just says “nobody will ever churn” is right 73.5% of the time, without doing anything at all. Yours gets 76%. It beats doing nothing by about two and a half points.

Say it before they do. “76% sounds alright until you know that always guessing ‘no’ gets you 73.5%. Accuracy was the wrong thing to measure. What actually matters is how many of the real churners I caught.”

Getting in front of this turns your weakest number into your best moment in the interview. If they find it first, you’re defending. If you raise it, you look like someone who checks their own work.

Q2.2 — Then why is accuracy on your resume?

“Because that’s the number the run actually recorded, and I wasn’t going to put something on my resume that I couldn’t show you in MLflow.”

That answer is worth more than a better number would have been. It says you don’t invent things.

Q2.3 — *What should you have measured instead?*

How many churners you caught (recall), how many of your churn predictions were right (precision), and the two combined (F1). Plus the ranking quality curves, ROC-AUC and PR-AUC.

For churn, catching them matters most. Missing a churning loses you the whole customer. A false alarm costs you one discount offer. So you'd pick the cutoff based on that cost difference, not leave it at the default 0.5.

The business percentages

Q2.4 — *Your resume says safety compliance rose 45%. How was that measured?*

Know exactly where that number came from — whose baseline, over what period, counted how. If it came from the client's own audit reports, say so plainly: "That's the facility's figure from their audit reports, not something I measured myself."

Attributing it honestly is completely fine. Implying you calculated it is not.

Q2.5 — *And the 25% audit-efficiency and 40% verification-time figures?*

Same rule. Before the interview, go through all four percentages and label each one: measured by me, reported by the client, or an estimate. Then say which when asked.

If you can't source one, take it off the resume. One number you can defend is worth more than four you can't.

The detection scores

Q2.6 — *0.841 mAP50 — what does that actually mean?*

In plain terms: it's a score out of 1 for how well the model finds things, averaged across your six classes.

For each class, you check every box the model drew. A box counts as correct if it overlaps the real object by at least half. You score the class on how many it found and how many it got wrong, then average across all six classes.

0.841 is a solid score for an industrial safety-gear dataset.

Q2.7 — *Why is your mAP50-95 only 0.467 when mAP50 is 0.841?*

Because the second number is much stricter about box placement. The first only asks "did you overlap the object by half?" The second asks the same question at steadily tighter overlaps, all the way up to near-perfect.

So: your model finds the right objects, but the boxes it draws are a bit loose. That's expected with small items like goggles and gloves, and with hand-drawn labels that aren't pixel-perfect either.

Q2.8 — *Which safety-gear class performed worst, and why?*

Have the per-class numbers in your head. It's usually goggles and gloves: they're the smallest things in the frame, they get hidden behind hands and heads most often, and there were fewer examples of them.

The fix: more pictures of them, a higher input resolution, or cutting the image into tiles so small objects appear bigger.

Q2.9 — Why YOLOv8m and not the small or large version?

Because of what you measured on your hardware. “The small one missed the little items. The large one couldn’t keep up across all the cameras on our GPU. Medium was the point where getting more accurate stopped being worth the extra time per frame.”

Any answer that’s a measured trade-off is a good answer.

Q2.10 — Why LangGraph for the rule assistant instead of plain LangChain?

Because the assistant isn’t one step — it loops. It works out what kind of violation happened, looks up candidate rules, checks whether those rules actually apply, and goes back to look again if they don’t.

A plain chain runs straight through once and can’t go back. LangGraph lets you build in the branch and the loop.

The honest gaps

Q2.11 — Your fire detector has no accuracy number. Why not?

“I tested it on data it hadn’t seen, but I don’t have a number I’d be willing to defend on a resume, so I left it off.”

Leaving something out on purpose reads as discipline. Never invent one under pressure — that’s the one mistake they won’t forgive.

Q2.12 — DocuMind answers in under 2.5 seconds. 2.5 seconds of what?

Know what that number includes. Is it the whole thing from question to answer, or just the search part? Was the model running locally or hosted? Was it a fresh start or already warmed up? How big was the document?

Say “that’s end to end on my machine, with the index already loaded.” Qualifying it is more credible than the bare number.

Q2.13 — Why 800-character chunks with 150 overlap?

Big enough to hold one complete idea, small enough that the search stays precise and you’re not paying for text you don’t need. The overlap means a sentence that falls across a boundary still appears whole in one of the two pieces.

Be honest if you picked it by feel rather than testing. Then say what test you’d run: try 400, 800 and 1600 and measure which one retrieves the right passage most often.

The uncomfortable ones

Q2.14 — Your Infosys dates are three months. How much of SafeSight was yours?

Answer precisely. Name the parts you built, name what other people built, and name what already existed before you arrived.

Claiming a whole team's system is the fastest way to fail this round, because the very next question is always a detail of whatever you just claimed.

Q2.15 — Your README says CI/CD is future work but your resume claims a pipeline. Which is true?

The resume. The workflow file exists and it builds the image and runs the import checks — the README is just out of date.

Then fix the README before you send the resume anywhere. And check the workflow actually triggers on the branch you use, or it has never run once.

Q2.16 — You list C++ but every project is Python. Where did you use C++?

Give the real scope and don't inflate it. "Coursework and algorithm practice. I'm comfortable reading it and writing algorithmic code in it. I haven't shipped a C++ system in production."

That's a perfectly good answer. Pretending otherwise is not.

Q2.17 — You list MCP and A2A agents. Show me where.

If no project uses them, say: "That's from my coursework and reading, not something I've shipped." Then be able to explain both properly.

A skill you can explain but haven't shipped is fine. One you can't explain is a lie sitting on a page.

Q2.18 — Which of your projects are you least proud of?

Pick one and explain why technically. The churn project is the honest choice: "The pipeline around it is the part I'd defend. The model itself needs a cutoff tuned to what a missed churning actually costs the business."

Q2.19 — Walk me through the hardest bug you hit.

The fire detector's email alerts. Sending an email takes a couple of seconds, and it was happening inside the loop that reads video frames — so the video stalled every time it sent one. You moved the email onto a separate thread so the video keeps running.

That's a good story: specific, you diagnosed it, you fixed it.

Q2.20 — If I clone your repo right now, will it run?

Know the answer before they ask. For each project: does it start from a clean machine with one command, are the model weights and settings documented, does the README describe what the code actually does?

If something's broken, say so first. They may well try it while you're talking.

PRACTICE | The hour that matters most

Go through your resume line by line and write one sentence next to every number explaining where it came from. Any number where you can't write that sentence should come off the resume. Then rehearse Q2.1 out loud until raising the 73.5% baseline yourself feels natural.

CHAPTER RECAP

- Raise the 73.5% churn baseline yourself. It is the single highest-value thing you can do in the whole interview.
- Label every business percentage as measured, client-reported, or estimated — and say which.
- mAP50 means “did it find the object”; mAP50-95 also grades how tightly the box fits.
- “I left it off because I couldn’t defend the number” is a strong answer.
- Never claim more of a team project than you built — the follow-up is always an implementation detail.
- Fix the stale READMEs before you send the resume anywhere.

PART II

The Programming Rounds

Python first, then the databases and the language you listed but have not shipped. Short answers. Say the sentence, then stop.

03 Python

04 SQL, C++ and MongoDB

Python

The thirty questions that come up in every screen

FOUNDATION

PREREQS: none

These appear in the phone screen and again in the technical round. The answers are short on purpose — one or two sentences each. Say the sentence, stop, and let them ask the follow-up.

WHAT YOU WILL LEARN

- Explain Python's core data types and when each is the right one
- Describe generators, decorators and context managers in plain words
- Answer the GIL question without reciting a paragraph
- Link an answer back to your own code wherever you can

Data types

Q3.1 — List vs tuple?

A list can be changed after you make it; a tuple can't. Because a tuple never changes, Python can use it as a dictionary key — a list can't be used that way.

Q3.2 — What is a dictionary, and how fast is a lookup?

It's a lookup table: you give it a name, it gives you back the thing. Looking something up takes the same tiny amount of time whether the dictionary holds ten items or ten million.

Q3.3 — List vs set?

A set has no duplicates, no order, and answering "is this in here?" is instant. With a list, Python has to walk through every item to check.

The rule: the moment you find yourself asking "is x in here?" repeatedly, you want a set.

Q3.4 — Mutable vs immutable types?

Mutable means changeable: lists, dictionaries, sets. Immutable means fixed once created: numbers, text, tuples. Fixed things are safe to share around, because nobody can change them behind your back.

Q3.5 — *Shallow vs deep copy?*

A shallow copy makes a new outer box but the things inside are still the same objects — change one and both “copies” change. A deep copy copies everything all the way down, so the two are genuinely independent.

Q3.6 — *What is a list comprehension?*

A compact way of building a list in one line: `[x*x for x in nums if x > 0]`. Shorter and faster than writing a loop that appends.

Q3.7 — *Generator vs list?*

A list builds everything up front and holds it all in memory. A generator hands you one item at a time and forgets it afterwards, so memory stays flat no matter how much data there is.

That’s why you use one to read a huge file or a video stream — you’d never fit it all in memory otherwise.

Q3.8 — *What does yield do?*

It pauses the function, hands one value back to whoever asked, and remembers exactly where it stopped. Next time you ask for a value, it carries on from there.

Functions and structure

Q3.9 — *What is a decorator?*

A function that wraps another one to add behaviour without editing it — adding logging, timing, or a login check. The `@app.get(...)` line in FastAPI is a decorator.

Q3.10 — **args and **kwargs?*

Ways of saying “accept however many extra arguments get passed.” `*args` collects the unnamed ones, `**kwargs` collects the named ones.

Q3.11 — *What is the GIL?*

A lock inside Python that means only one thread can run Python code at a time.

The practical consequence: threads help when your code is **waiting** for something — a network call, a file, sending an email. They don’t help when your code is **calculating** something, because only one thread gets to run anyway. For heavy calculation you need separate processes.

This is exactly why the threaded email alerts in your fire detector work. Sending mail is waiting, not calculating.

Q3.12 — *Multithreading vs multiprocessing?*

Threads share memory and are cheap to create, but the lock above means they only help with waiting. Processes each get their own memory and really do run at the same time, so they help with calculation — but they cost more to start.

Q3.13 — `is` vs `==`?

`==` asks “are these two things equal?” `is` asks “are these literally the same object in memory?” Only use `is` when checking for `None`.

Q3.14 — *How does Python manage memory?*

It counts how many things are pointing at each object, and throws an object away when nothing points at it any more. There’s also a cleanup pass for objects that point at each other in a circle, which the counter alone can’t catch.

Q3.15 — *What is a lambda?*

A tiny unnamed function written in one line, usually passed straight to something else: `sorted(d, key=lambda x: x[1])`.

Q3.16 — `map`, `filter`, `reduce`?

`map` runs a function over every item. `filter` keeps the items that pass a test. `reduce` squashes the whole sequence down to one value. In practice a comprehension is usually easier to read.

Q3.17 — *What does `enumerate` give you?*

The position and the item together, so you don’t have to keep a counter yourself: `for i, frame in enumerate(frames)`.

Q3.18 — *What does `zip` do?*

Pairs up items from two or more lists, side by side, stopping when the shortest one runs out. `zip(boxes, labels)`.

Errors, classes and files

Q3.19 — *How do you handle exceptions?*

`try` the risky thing, `except` the specific error you expect, and `finally` for cleanup that must happen either way.

Catch the specific error, not everything. A bare `except` swallows real bugs and you never find out.

Q3.20 — *What is a context manager?*

Something you use with `with`, which guarantees cleanup happens. `with open(f) as fh:` closes the file even if the code inside crashes.

Q3.21 — *What are the four pillars of object-oriented programming?*

Keep an object’s internals private (encapsulation). Build new classes on top of old ones (inheritance). Let different classes respond to the same method name in their own way (polymorphism). Show a simple interface and hide the messy mechanics behind it (abstraction).

Q3.22 — `__init__` vs `__new__`?

`__new__` creates the object; `__init__` fills it in. You almost always only ever write `__init__`.

Q3.23 — What is `self`?

The particular object the method was called on. Python passes it in as the first argument, explicitly, rather than hiding it.

Q3.24 — Class method vs static method vs instance method?

An instance method works on one object and takes `self`. A class method works on the class itself and is often used as an alternative way to build one. A static method takes neither — it's just a plain function kept inside the class for tidiness.

Q3.25 — Module vs package?

A module is one `.py` file. A package is a folder of them.

Q3.26 — What are virtual environments, and why use them?

A separate set of installed libraries for each project, so one project's PyTorch version can't break another project. `venv`, `conda`, or `uv`.

Q3.27 — How do you read a large file without running out of memory?

Loop over it line by line rather than loading the whole thing: `for line in open(path)`. For a big CSV, read it in chunks with `pd.read_csv(..., chunksize=...)`.

Q3.28 — What is `if __name__ == "__main__":` for?

It runs that block only when you run the file directly, not when some other file imports it. Without it, importing a module would fire off its script behaviour as a side effect.

Q3.29 — What is type hinting, and does it slow anything down?

Notes in the code saying what type each argument should be. Python ignores them when running, so there's no cost. Tools read them — which is how FastAPI turns them into automatic request checking.

Q3.30 — `append` vs `extend`?

`append` adds one item. `extend` adds every item from a list, one by one. `[1,2].append([3,4])` gives you `[1,2,[3,4]]`; `extend` gives you `[1,2,3,4]`.

PRACTICE | One thing to rehearse

The GIL question (Q3.11) is the one most likely to come up and most likely to be answered badly. Practise saying it in two sentences: only one thread runs Python at a time, so threads help with waiting and not with calculating. Then land it on your fire detector's email thread.

CHAPTER RECAP

- Tuple, set and dictionary each exist for one reason — know which reason.
- A generator hands you one item at a time, which is why it handles files bigger than memory.
- The GIL means threads help with waiting, processes help with calculating.
- `is` is only for `None`.
- Catch specific exceptions, never a bare `except`.
- Where an answer can land on your own code, land it there.

SQL, C++ and MongoDB

The database round, and the language you listed

FOUNDATION

PREREQS: none

Almost every data or ML interview includes a live SQL question, so practise writing it rather than recognising it. C++ and MongoDB are on your resume, which makes them fair game — keep those answers short and do not overclaim depth.

WHAT YOU WILL LEARN

- Write the common SQL patterns without looking them up
- Explain joins, grouping and window functions in plain words
- Answer C++ questions honestly without inflating your experience
- Say when you would pick MongoDB over a normal database

SQL: the basics

Q4.1 — What are the main join types?

A join glues two tables together on a matching column.

Inner keeps only rows that exist in both. **Left** keeps everything from the left table, filling blanks where there's no match. **Right** does the same from the other side. **Full outer** keeps everything from both. **Cross** pairs every row with every row, which is almost never what you want.

Q4.2 — WHERE vs HAVING ?

WHERE filters individual rows before they're grouped. **HAVING** filters the groups after they've been totalled up. "Only customers in Mumbai" is **WHERE**; "only groups with more than five orders" is **HAVING**.

Q4.3 — What does GROUP BY do?

Squashes all the rows that share a value into one row, so you can total or average over each group. One row per contract type, say, instead of one row per customer.

Q4.4 — Name the aggregate functions.

COUNT, **SUM**, **AVG**, **MIN**, **MAX**. One catch: **COUNT(column)** skips blanks, **COUNT(*)** counts every row.

Q4.5 — DELETE vs TRUNCATE vs DROP ?

DELETE removes the rows you pick and can be undone. **TRUNCATE** empties the whole table quickly. **DROP** removes the table itself.

Q4.6 — Primary key vs unique key?

Both stop duplicates. A primary key can never be blank and there's only one per table. A unique key can be blank and you can have several.

Q4.7 — What is a foreign key?

A column pointing at another table's primary key. It stops you from, say, creating an order for a customer who doesn't exist.

Q4.8 — What is an index, and what does it cost?

Like the index at the back of a book — it lets the database find rows without reading every page. The cost is that writing becomes slower and it takes up space, so you don't index everything.

Q4.9 — Clustered vs non-clustered index?

A clustered index decides the actual physical order the rows are stored in, so there's only one. A non-clustered index is a separate lookup list pointing at the rows.

Q4.10 — UNION vs UNION ALL ?

UNION stacks two result sets and removes duplicates, which means it has to sort. **UNION ALL** just stacks them and is faster. Use **ALL** unless you actually need the deduplication.

Q4.11 — Subquery vs CTE?

Both are a query nested inside another. A CTE — the **WITH x AS (...)** form — gets a name, so it's readable and you can refer to it more than once.

SQL: the parts that get asked live**Q4.12 — What are window functions?**

Calculations across a group of rows that **don't** collapse them into one row. So you can show each employee alongside their rank in their department, with every employee still listed.

```
ROW_NUMBER() OVER (PARTITION BY dept ORDER BY salary DESC) .
```

Q4.13 — ROW_NUMBER vs RANK vs DENSE_RANK ?

With a tie: **ROW_NUMBER** gives 1, 2, 3 — it just numbers them. **RANK** gives 1, 1, 3 — ties share a place and then it skips. **DENSE_RANK** gives 1, 1, 2 — ties share and it doesn't skip.

Q4.14 — Find the second-highest salary.

SELECT MAX(salary) FROM emp WHERE salary < (SELECT MAX(salary) FROM emp) . Or use **DENSE_RANK() = 2**, which handles ties the way most people mean.

Q4.15 — Find duplicate rows.

`SELECT col, COUNT(*) FROM t GROUP BY col HAVING COUNT(*) > 1`. Group by the thing, keep the groups bigger than one.

Q4.16 — How do you handle blanks (NULLs)?

Test them with `IS NULL` or `IS NOT NULL`. Replace them with `COALESCE(col, 0)`.

The trap: a blank is not equal to anything, including another blank. So `NULL = NULL` is not true, which is why you can't use `=` to test for it.

Q4.17 — What is normalisation? Name the first three forms.

Splitting data across tables so nothing is stored twice. First form: one value per cell, no lists crammed into a field. Second: every column depends on the whole key, not part of it. Third: no column depends on another non-key column.

Q4.18 — When would you denormalise?

When you read far more than you write, and joining tables every time costs more than storing a bit of duplicate data. Common in analytics.

Q4.19 — What are ACID properties?

The four guarantees a database gives a transaction: it either fully happens or not at all, it leaves the data valid, parallel transactions don't corrupt each other, and once it's confirmed it survives a crash.

Q4.20 — What does EXPLAIN tell you?

The plan the database is about to follow — which indexes it'll use, what order it'll join in, how many rows it expects. It's the first thing you look at when a query is slow.

Q4.21 — DISTINCT vs GROUP BY?

Both remove duplicates. `GROUP BY` also lets you calculate something per group, which `DISTINCT` can't.

Q4.22 — Get the churn rate by contract type from your telecom table.

```
1 SELECT contract,
2     AVG(CASE WHEN churn = 'Yes' THEN 1.0 ELSE 0 END) AS rate
3 FROM customers
4 GROUP BY contract
5 ORDER BY rate DESC;
```

Expect a resume-linked question exactly like this. The trick is turning Yes/No into 1/0 so you can average it.

C++**Q5.1 — Why is C++ used in ML at all?**

Speed and control. PyTorch, OpenCV and the YOLO runtimes are all C++ underneath, with Python wrappers on top. You write Python; C++ does the work.

Q5.2 — C vs C++?

C++ adds classes, templates, exceptions and a big standard library on top of C's simpler procedural core.

Q5.3 — What is a pointer? What is a reference?

A pointer holds the address of something. You can point it somewhere else, or at nothing. A reference is another name for an existing thing — it's attached once and can never be empty.

Q5.4 — Stack vs heap?

Stack memory is handled for you and disappears when the function ends — fast and small. Heap memory you ask for and must give back yourself — bigger, and it lasts until you release it.

Q5.5 — What causes a memory leak, and how do you avoid it?

Asking for heap memory and never giving it back. Avoid it by using smart pointers, which release the memory automatically when nothing needs it any more.

Q5.6 — What is RAII?

Tying a resource's lifetime to an object. When the object goes out of scope, its cleanup runs automatically — so a file gets closed or memory gets freed without you remembering to do it.

Q5.7 — What is the STL?

The standard library of ready-made containers (`vector`, `map`, `set`) and algorithms (`sort`, `find`), so you aren't writing those yourself.

Q5.8 — `vector` vs `array`?

A vector grows and shrinks as you add things. A plain array is a fixed size decided up front.

Q5.9 — `map` vs `unordered_map`?

`map` keeps things sorted and lookups take slightly longer. `unordered_map` is a hash table — no order, but lookups are instant.

Q5.10 — Overloading vs overriding?

Overloading: several functions with the same name but different arguments, and the compiler picks. Overriding: a child class replaces a method it inherited, and the choice happens while the program runs.

Q5.11 — What is a virtual function?

A method where the version that runs is decided by what the object actually is, not by what type the variable claims to be. It's what makes polymorphism work.

Q5.12 — What are templates?

A way to write one function or class that works for any type, so you don't write the same thing again for `int`, `float` and everything else.

Q5.13 — Pass by value vs by reference?

By value copies the whole argument. By reference hands over the original, so nothing gets copied. Use a `const` reference for big things you only need to read.

WATCH OUT | Keep the C++ answers short

You listed it, so you must be able to answer. But do not volunteer depth you do not have. “I’m comfortable reading it and writing algorithmic code; I haven’t shipped a C++ system” closes the topic cleanly and honestly.

MongoDB

Q6.1 — What is MongoDB?

A database that stores documents instead of rows. Each document is basically a JSON object, and two documents in the same place don’t have to have the same fields.

Q6.2 — SQL vs MongoDB — when do you pick which?

A normal database when your data is structured, has real relationships, and you need those relationships enforced. MongoDB when your data is nested or its shape keeps changing, or when you need to spread it across many machines.

Pick by the shape of your data, not by which one sounds more modern.

Q6.3 — Map the vocabulary: table, row, column?

Collection, document, field.

Q6.4 — What is BSON?

MongoDB’s storage format — basically JSON in a binary form, with extra types JSON doesn’t have, like proper dates.

Q6.5 — What is `_id`?

The unique identifier every document must have. If you don’t supply one, MongoDB generates one.

Q6.6 — How do you query documents?

`db.users.find({ city: "Mumbai" })`. You pass a second argument if you only want some of the fields back.

Q6.7 — What is the aggregation pipeline?

A series of stages that reshape your data step by step — filter, group, sort, rename. It’s MongoDB’s version of `GROUP BY` and joins.

Q6.8 — Does MongoDB support joins?

Yes, with `$lookup` inside an aggregation, but it’s heavier than a normal database join. Often you nest the data inside the document instead.

Q6.9 — Embedding vs referencing?

Embed the child data inside the parent when you always read them together and it stays small. Point at it from outside when it's large, shared by several parents, or keeps growing.

Q6.10 — What is sharding? What is replication?

Sharding splits your data across several machines so you can hold more than one machine can. Replication keeps copies on other machines so you don't lose anything if one dies.

Q6.11 — Does MongoDB have transactions?

Yes, across multiple documents since version 4.0. A write to a single document was always all-or-nothing anyway.

Q6.12 — What is indexing in MongoDB?

Same idea as SQL — a lookup structure that makes searches fast. `db.c.createIndex({ email: 1 })`, and `.explain()` to check it's being used.

Q6.13 — What is the CAP theorem, and where does MongoDB sit?

When the network between your machines breaks, you have to choose: keep every machine giving the same answer, or keep every machine answering at all. You can't have both.

MongoDB chooses consistency by default — it would rather refuse than give you stale data. You can tune that per query.

CHAPTER RECAP

- A join combines tables; **WHERE** filters rows, **HAVING** filters groups.
- Window functions calculate across rows without collapsing them — the most likely live SQL question.
- **NULL** is never equal to anything, including **NULL**.
- Keep C++ answers short and honest: reading and algorithms, not production.
- Pick MongoDB for nested or changing data shapes, not because it sounds modern.

PART III

Data and Classical ML

Statistics, handling real data, the classic algorithms, and how you prove a model is any good. The most reliably asked block in the whole interview.

-
- 05 Statistics
 - 06 Pandas, NumPy and Looking at Data
 - 07 Classical Machine Learning
 - 08 Imbalanced Data and Metrics

Statistics

The dozen ideas you actually need, explained without the maths

FOUNDATION

PREREQS: none

Your BITS coursework lists statistical modelling, so this gets probed. Short definitions are enough — nobody wants a derivation. The ones worth real care are p-values, confidence intervals and the ways a sample can lie to you, because those are where most people say something wrong.

WHAT YOU WILL LEARN

- Define the common statistics without reaching for formulas
- Say what a p-value is and, more importantly, what it is not
- Name the ways a sample can mislead you
- Connect a statistics answer back to your own data

Describing data

Q7.1 — Mean vs median vs mode — when do you use the median?

When a few extreme values would drag the average somewhere silly. If nine people earn £30k and one earns £3 million, the average says £327k and the median says £30k. The median describes the group better.

Income and response times are the classic cases.

Q7.2 — Variance vs standard deviation?

Both measure how spread out the data is. Standard deviation is the useful one because it's in the same units as your data — if you're measuring heights in centimetres, the standard deviation is in centimetres too.

Q7.3 — What is skewness? What is kurtosis?

Skewness is lopsidedness. A long tail stretching to the right is positive skew — income does this. Kurtosis is about how heavy the tails are, meaning how often you get extreme values.

Q7.4 — What is the normal distribution, and why is it everywhere?

The bell curve. It turns up constantly because when lots of small independent things add together, the result tends to look like a bell curve regardless of what the individual things looked like.

Q7.5 — State the Central Limit Theorem.

If you take lots of samples and average each one, those averages form a bell curve — even if the original data looked nothing like one. That’s what makes confidence intervals possible at all.

Q7.6 — What is the law of large numbers?

The more you measure, the closer your average gets to the true average. It’s why a hundred coin flips lands nearer to 50/50 than ten flips does.

Testing and uncertainty

Q7.7 — What is a *p*-value, in plain words?

It answers one question: **if there were really no effect, how often would I see a result at least this striking by pure chance?** A small *p*-value means “this would be a surprising fluke.”

What it is **not**: the probability that there’s no effect. That’s the mistake almost everyone makes, and interviewers listen for it.

Q7.8 — Type I vs Type II error?

Type I is a false alarm — you say something’s there when it isn’t. Type II is a miss — you say nothing’s there when it is.

In your fire detector, a Type I error is an unnecessary email. A Type II error is a fire nobody was told about.

Q7.9 — What is a confidence interval?

A range of plausible values rather than a single number. “95% confidence” means the method you used produces a range that contains the truth 95 times out of 100.

It’s about the method’s track record, not about this one particular range — that’s the subtlety people get wrong.

Q7.10 — What is a hypothesis test? Name two.

A procedure for deciding whether your data is strange enough to stop believing “nothing is going on.” A *t*-test compares two averages. A chi-square test checks whether two categories are related.

Q7.11 — *T*-test vs *z*-test?

Use a *z*-test when you have a lot of data or you already know the spread of the whole population. Use a *t*-test when you have a small sample and you’re estimating the spread from that sample. In practice it’s nearly always a *t*-test.

Q7.12 — What is ANOVA?

A test for whether three or more groups have different averages. It exists so you don't run lots of separate two-group tests, which would throw up false positives just by chance.

Q7.13 — What is a chi-square test used for?

Checking whether two categorical things are related — like whether contract type is connected to whether someone churns. Which is directly relevant to your churn data.

Q7.14 — Correlation vs covariance?

Both describe whether two things move together. Covariance gives you the direction but in awkward units. Correlation rescales it to between -1 and $+1$, so you can compare across different measurements.

Q7.15 — Correlation vs causation — how do you establish causation?

Correlation alone never establishes it. To show cause you need to intervene — run an experiment where you change one thing at random and see what happens. Without that, you're guessing.

Q7.16 — What is Bayes' theorem?

A rule for updating what you believe when new evidence arrives. You start with how likely something was, you see some evidence, and you end up with a revised belief.

The classic example: a test that's 99% accurate for a disease only one in ten thousand people have will still give you mostly false alarms, because there are so many more healthy people to get wrong.

Q7.17 — Probability vs likelihood?

Probability goes forward: given the rules, how likely is this outcome? Likelihood goes backward: given this outcome, which set of rules explains it best?

Q7.18 — What is maximum likelihood estimation?

Picking the settings that make the data you actually saw as unsurprising as possible. It's what logistic regression is doing when it trains.

The ways data misleads

Q7.19 — What is an outlier, and how do you detect one?

A point that sits far from everything else. Find them with a box plot, or by flagging anything more than about one and a half times the middle spread away from the middle.

Don't just delete them. Fraud cases and equipment failures **are** the outliers — they're often the thing you're trying to find.

Q7.20 — Population vs sample?

The population is everyone you want to describe. The sample is the subset you actually measured. Every statistic you compute is from the sample and is only a guess about the population.

Q7.21 — What is sampling bias? Give an example from your own work.

When your sample doesn't represent the thing you care about.

Your own example: if all your safety-gear training footage was filmed during daytime shifts, the detector will quietly get worse at night — and your test set, filmed the same way, won't warn you.

Q7.22 — Selection bias vs survivorship bias?

Selection bias: who ended up in your sample wasn't random. Survivorship bias: you only see the ones that lasted.

The churn version: if you only study customers who are still around, you've removed exactly the people you're trying to learn about.

Q7.23 — What is expected value?

The average outcome if you repeated something many times, weighted by how likely each outcome is. You use it to put an actual number on a decision — like whether a 10% chance of losing a customer justifies a discount offer.

Q7.24 — What does *i.i.d.* mean, and when is it violated?

It means every data point is independent of the others and drawn from the same source. Most methods quietly assume it.

It's violated by anything with time in it — and by consecutive video frames, which are nearly identical to each other. **That's why you must split video data by clip, not by frame.** Split by frame and nearly identical pictures end up in both training and testing, so your score comes out far too good.

PRACTICE | The one to rehearse

Q7.7 — the p-value. Say it as: "If there was really no effect, how often would I see something this striking by chance?" Then say what it isn't. Getting that distinction right marks you out immediately.

CHAPTER RECAP

- Use the median when extreme values would drag the average somewhere silly.
- A p-value is how surprising your result would be if nothing were going on — not the chance that nothing is going on.
- A confidence interval describes the method's track record, not one range.
- Correlation never proves cause; only an experiment does.
- Don't delete outliers blindly — they are often the thing you're looking for.
- Consecutive video frames are not independent, which is why you split by clip.

Pandas, NumPy and Looking at Data

The round where a live task catches people out

FOUNDATION

PREREQS: none

Your resume lists all of this, and it is the easiest place to be caught by a practical task. The two answers that matter most are the one about data leakage and the one about your own imputation choice — both show whether you actually looked at your data or just ran code on it.

WHAT YOU WILL LEARN

- Handle missing values with a reason, not a reflex
- Explain why vectorised code is faster in one sentence
- Describe your own EDA as an ordered process
- Spot data leakage before it reaches a model

The basics

Q8.1 — DataFrame vs Series?

A Series is one column with labels. A DataFrame is a whole table made of Series side by side.

Q8.2 — loc vs iloc?

`loc` picks by name — “give me the row called ‘Mumbai’”. `iloc` picks by position — “give me the third row”.

Q8.3 — How do you handle missing values?

First count them: `df.isna().sum()`. Then choose per column.

Drop the rows if there are very few. Fill them in if there are more — use the middle value for skewed numbers, the most common value or an explicit “Unknown” for categories. And if **being missing is itself informative**, add a separate column recording that it was missing, because that fact may be the signal.

Q8.4 — Which imputation did you use on the churn data, and why?

Be specific here — it’s a good question for you. In that dataset, the `TotalCharges` field is blank for brand-new customers who’ve been there zero months. Those aren’t randomly missing values; they’re genuinely zero.

Knowing that detail is strong evidence you actually opened the data instead of just running `fillna`.

Q8.5 — `merge` vs `join` vs `concat`?

`merge` combines two tables on matching columns, like SQL. `join` does the same but on the row labels. `concat` just stacks tables on top of each other or side by side.

Q8.6 — What does `groupby` do?

Split the rows into groups, do something to each group, put the answers back together. `df.groupby('contract')['churn'].mean()` gives you the churn rate for each contract type.

Q8.7 — `apply` vs *vectorised operations*?

`apply` runs your function once per row in Python, which is slow. A vectorised operation hands the whole column to compiled code that does it in one go — often fifty to a hundred times faster.

The rule: if you're writing `apply` on a big table, there's almost always a built-in way that's much faster.

Q8.8 — What is a *pivot table* in pandas?

A cross-tabulation — rows down one side, categories across the top, a calculation in each cell. `df.pivot_table(index=..., columns=..., values=..., aggfunc=...)`.

Q8.9 — How do you find and drop duplicates?

`df.duplicated().sum()` to count them, `df.drop_duplicates()` to remove them.

Q8.10 — What does `df.describe()` give you, and what does it hide?

Count, average, spread, minimum, maximum and the quartiles for your number columns.

What it hides matters more: it skips the text columns entirely, it tells you nothing about where the missing values are, and it says nothing about the **shape** of the data. Two completely different distributions can have the same average and spread. Always plot as well.

NumPy

Q8.11 — What is a NumPy array, and why is it faster than a list?

A solid block of memory where every item is the same type. That lets the operation run in compiled code across the whole block, rather than Python looping over items one at a time.

Q8.12 — What is *broadcasting*?

NumPy automatically stretching a smaller array to match a bigger one so the shapes line up. Subtracting one row of averages from a whole table works without you writing a loop.

Q8.13 — reshape vs ravel vs transpose?

`reshape` changes the dimensions, `ravel` flattens everything into one long line, `transpose` swaps the axes round. Usually none of them copies the data — you get a different view of the same memory.

Q8.14 — View vs copy in NumPy — why does it matter?

A view shares the same memory as the original. So if you change the view, you have silently changed the original too.

That's a real source of bugs. Use `.copy()` when you actually mean to make an independent one.

Looking at the data

Q8.15 — What steps does your EDA actually follow?

Have an ordered answer ready — it's much better than listing plot types.

Shape and column types first. Then where the missing values are. Then how balanced the target is. Then the distribution of each column on its own. Then how each column relates to the target. Then outliers and a leakage check.

Q8.16 — Which plot for which question?

Histogram for the shape of one column. Box plot for spread and outliers. Scatter for two numbers against each other. Bar chart for counting categories. Heatmap for a grid of correlations. Line for anything over time.

Q8.17 — What is data leakage, and how do you catch it?

When information that wouldn't be available at prediction time sneaks into training. The model scores brilliantly in testing and then fails completely in the real world, because in the real world that information doesn't exist yet.

Catch it by asking of every single column: **would I actually know this before the thing I'm predicting happens?** And by fitting your scalers and encoders inside each training fold, not on the whole dataset before splitting.

Q8.18 — How do you encode categorical variables?

One-hot when there are only a few categories — one yes/no column each. Ordinal numbering when the order is real, like small/medium/large. For categories with hundreds of values, replace each with a summary number — but fit that inside the training fold or you've just leaked the answer.

Q8.19 — Why scale features? Which models need it?

Because some models measure distance, and a column in rupees will dominate a column measured 0 to 1 purely because the numbers are bigger.

Needed by: SVM, KNN, k-means, logistic regression, neural networks. Not needed by trees and random forests — they only care about the order of values, not the size.

Q8.20 — *Standardisation vs normalisation?*

Standardisation shifts and stretches so the average is 0 and the spread is 1. Normalisation squashes everything into a fixed range, usually 0 to 1.

Standardise when the method expects roughly bell-shaped inputs. Normalise when you need a hard range.

PRACTICE | The leakage habit

Take the churn dataset and go column by column asking “would I know this before the customer churned?” Write down any column where the answer is no or unclear. That is the habit the question in Q8.17 is testing for, and doing it once makes the answer sound lived rather than learned.

CHAPTER RECAP

- Missing values deserve a reason per column, and “it’s missing” can itself be a signal.
- Your **TotalCharges** answer is a genuinely strong one — it proves you looked.
- **apply** loops in Python; vectorised operations don’t, which is the whole speed difference.
- A NumPy view shares memory with the original — changing it changes both.
- Describe your EDA as an ordered process, not a list of charts.
- Leakage is the one that fails a model in production; ask “would I know this yet?” of every column.

Classical Machine Learning

The algorithms, explained the way you would to a colleague

FOUNDATION

PREREQS: none

Your resume names scikit-learn, SVM and Random Forest, so all of this is mandatory. Every answer here is one or two sentences plus what it is good for — that second part is what separates someone who has used the algorithm from someone who has read about it.

WHAT YOU WILL LEARN

- Explain overfitting and the bias-variance trade in plain words
- Describe each algorithm in two sentences and say when to use it
- Justify why you chose Random Forest, and what you would try next
- Answer the cross-validation questions without reciting

The core ideas

Q9.1 — Supervised vs unsupervised vs reinforcement learning?

Supervised: you show it examples with the right answers attached, and it learns to produce them. Unsupervised: no answers, it just finds structure — groups, patterns. Reinforcement: it tries things and learns from reward and punishment.

Q9.2 — Classification vs regression?

Classification picks a category — will this customer churn, yes or no. Regression predicts a number — how much will they spend.

Q9.3 — What is the bias-variance tradeoff?

Two different ways a model gets things wrong.

Bias is being too simple to capture the pattern — like drawing a straight line through data that curves. It gets things wrong consistently.

Variance is being too sensitive to the exact examples you trained on — it memorises instead of learning, so a slightly different dataset gives a very different model.

Making a model more complex reduces one and increases the other. The best model sits somewhere in between.

Q9.4 — Overfitting vs underfitting — how do you spot each?

Overfitting: scores brilliantly on the training data, badly on anything new. It memorised.

Underfitting: scores badly on both. It never learned.

You spot them by always comparing the two numbers. Looking only at the training score tells you nothing.

Q9.5 — How do you fix overfitting?

More data, if you can get it. Otherwise: make the model simpler, add a penalty for complexity, stop training earlier, use cross-validation to catch it, and for trees, limit how deep they're allowed to grow.

Q9.6 — What is regularisation? L1 vs L2?

A penalty added for having large weights, which pushes the model toward simpler solutions.

L1 pushes some weights to exactly zero, so it effectively deletes features. L2 shrinks all of them smoothly without removing any. Use L1 when you want to find out which features matter; L2 when you just want to calm things down.

Q9.7 — What is cross-validation? Why k-fold?

Instead of one train/test split, you cut the data into five parts, train on four and test on the fifth, then rotate so every part gets a turn being the test set.

Why: one split can be lucky or unlucky. Five give you an average and a sense of how much the number wobbles.

Q9.8 — What is stratified k-fold, and why would you need it for churn?

Stratified means each fold keeps the same mix of classes as the whole dataset.

With only 26.5% churners, a plain random split could easily hand you a fold with very few churners in it — and then your recall number is measured on almost nothing and bounces around meaninglessly.

Q9.9 — Train, validation, test — what's each for?

Train: the model learns from it. Validation: you use it to choose settings and compare models. Test: you touch it **once**, at the very end, for an honest final number.

The moment you start tuning against the test set, it stops being a test.

The algorithms

Q9.10 — Explain linear regression.

It draws the straight line that sits closest to all your points, and uses that line to predict. Simple, fast, and you can read exactly what it's doing from the coefficients.

Q9.11 — Explain logistic regression. Is it linear?

It takes a straight-line calculation and squashes the result into a number between 0 and 1, which you read as a probability.

Is it linear? The boundary it draws between the two classes is a straight line. The output isn't — it curves, because of the squashing.

Q9.12 — Why not use linear regression for classification?

Because its output isn't bounded, so it'll happily predict a "probability" of 1.4 or -0.2, which is meaningless. The squashed version doesn't have that problem.

Q9.13 — Explain decision trees. How do they split?

A tree of yes/no questions. "Is the contract month-to-month? Yes → is tenure under 6 months? Yes → likely to churn."

At each step it tries every possible question and picks the one that best separates the classes. Left to grow fully, a tree will memorise the training data completely.

Q9.14 — Gini vs entropy?

Two ways of measuring how mixed-up a group is. Entropy uses logarithms and costs slightly more to compute. In practice they produce nearly identical trees, so it rarely matters.

Q9.15 — Explain random forest. Why is it better than one tree?

Lots of trees, each trained on a different random sample of the data and allowed to look at only a random subset of the columns. Then they vote.

Why that helps: each tree is wrong in its own way, and when you average lots of **differently** wrong answers, the errors largely cancel out. One tree alone would memorise.

Q9.16 — Your churn model uses 300 trees. Why 300?

Adding more trees to a forest never makes it overfit — it just stabilises. So you raise the number until the score stops improving, then stop paying for the extra compute.

Be honest if 300 was a sensible default rather than something you tuned. That's a perfectly fine answer.

Q9.17 — Bagging vs boosting?

Bagging builds lots of models in parallel on different samples and averages them. It fixes the "too sensitive" problem.

Boosting builds models one after another, each one focusing on what the previous ones got wrong. It fixes the "too simple" problem.

Q9.18 — What is gradient boosting? Name an implementation.

Each new tree is trained specifically on the mistakes the current set of trees is still making. XGBoost, LightGBM and CatBoost are the implementations.

On tabular data it usually beats random forest — worth saying you'd try it on the churn problem.

Q9.19 — Explain SVM.

It finds the dividing line that leaves the widest possible gap between the two classes. Only the points nearest the boundary matter — everything far away is ignored.

Q9.20 — What is the kernel trick?

A way of drawing a curved boundary without actually doing the expensive work of moving your data into a higher dimension. It gets the same answer much more cheaply.

Q9.21 — What do C and γ do in an SVM?

C controls how much you care about getting every training point right versus keeping a wide clean gap. High C means fitting tightly, which risks memorising.

γ controls how far each training point's influence reaches. High γ means each point only affects its immediate neighbourhood.

Q9.22 — Explain KNN. What's its weakness?

To classify something, look at the handful of training examples closest to it and go with the majority.

Weaknesses: there's no training step, so all the cost is at prediction time and it's slow on big data. It needs scaled features. And it falls apart when you have many columns, because everything ends up roughly equally far from everything else.

Q9.23 — Explain Naive Bayes. What's "naive" about it?

It uses Bayes' rule to work out which class is most likely given the features.

The naive part: it assumes every feature is independent of all the others, which is almost never true. Oddly it works very well anyway, especially on text.

Q9.24 — Explain k -means. How do you choose k ?

Pick k starting points, assign every data point to its nearest one, move each centre to the middle of its group, repeat until nothing moves.

Choose k by plotting how much the clusters improve as you add more — you look for the point where the curve bends and extra clusters stop helping much.

Q9.25 — What are k -means' limitations?

You have to decide the number of groups up front. It assumes the groups are roughly round and roughly the same size. It's sensitive to where you start. And it cares about scale, so you must standardise first.

Q9.26 — What is hierarchical clustering? And DBSCAN?

Hierarchical repeatedly merges the two closest groups, building a tree you can cut at any level.

DBSCAN groups by density — wherever points are packed closely together, that's a cluster. It finds odd shapes, labels isolated points as noise, and you don't have to pick the number of groups in advance.

Q9.27 — Explain PCA.

It finds the directions in which your data varies most, and describes each point using just those few directions. You keep most of the information with far fewer columns.

The cost: the new columns are blends of the old ones, so you can no longer say "this is age" about any of them.

Q9.28 — What is the curse of dimensionality?

As you add more columns, your data spreads thinner and thinner, and the distance between any two points starts looking about the same. Anything based on “which points are nearest” stops working.

Q9.29 — Feature selection vs feature extraction?

Selection keeps some of your original columns and throws the rest away. Extraction builds new columns out of combinations of the old ones, like PCA does.

Q9.30 — What is feature importance in a random forest? What’s the catch?

It measures how much each column helped separate the classes across all the trees.

Two catches: it unfairly favours columns with lots of distinct values, and when two columns say the same thing it splits the credit between them so both look unimportant. Permutation importance or SHAP is more trustworthy.

Q9.31 — Hyperparameter vs parameter?

Parameters are what the model learns from the data. Hyperparameters are the settings you choose before training — tree depth, learning rate, the number of neighbours.

Q9.32 — How do you tune hyperparameters?

Try every combination if there are only a few. Try random combinations if there are many — it’s surprisingly more effective than a grid. Use a smarter search like Optuna when each run is expensive.

Always tune against validation folds. Never against the test set.

Q9.33 — What is an ensemble? Name three ways to build one.

Combining several models so the group beats any individual. Bagging (average many parallel models), boosting (build them in sequence fixing mistakes), and stacking (train another model to decide how to combine them).

Q9.34 — What is the no-free-lunch theorem?

No single algorithm is best at everything. Which is why you benchmark a few rather than assuming your favourite will win.

Q9.35 — How would you choose a model for a new tabular problem?

Start with something trivial as a baseline — predict the most common class. Then logistic regression, because it’s fast and you can read it. Then gradient boosting, because it usually wins on tabular data.

Compare them on the same splits using a metric tied to what the business actually cares about, not whichever number looks best.

PRACTICE | The one to prepare

Q9.16 and Q9.4 together. You should be able to say why 300 trees is a safe choice, and how you’d show whether your model is overfitting — comparing training score against validation score. Those two are asked constantly.

CHAPTER RECAP

- Bias is too simple, variance is too sensitive; complexity trades one for the other.
- Always compare training score against validation score — one alone tells you nothing.
- A random forest works because its trees are wrong in different ways.
- More trees never overfit a forest; more trees eventually hurt a boosted model.
- Stratified folds matter for churn because a plain split can leave you with almost no churners to measure.
- Tune against validation, never against test.

Imbalanced Data and Metrics

The most reliably asked block in the whole interview

FOUNDATION

PREREQS: Chapter 7

Your resume lists imbalanced data handling and your churn set is 26.5% minority, so this is coming. Know precision and recall cold — which way round they go, and which one matters for each of your projects. These are the questions where a confident wrong answer does real damage.

WHAT YOU WILL LEARN

- Explain why accuracy misleads, using your own numbers
- Say precision and recall correctly, every time
- Choose the right metric for each of your four projects
- Describe SMOTE and the mistake everybody makes with it

Imbalanced data

Q10.1 — What is class imbalance, and why is it a problem?

When one outcome is much rarer than the other. The problem is that a model can score brilliantly by simply never predicting the rare one — and the rare one is almost always the thing you care about.

Q10.2 — Your churn data is 26.5% minority. Is that even imbalanced?

Mildly. Roughly one in four, which is a long way from fraud detection at one in a thousand. So heavy resampling probably isn't needed.

The real problem isn't the imbalance — it's that accuracy was the wrong metric at **any** level of imbalance. Saying that shows judgement rather than reaching for SMOTE by reflex.

Q10.3 — Which metrics do you use instead of accuracy?

Precision, recall and F1 on the class you care about, plus the ranking curves. When the rare class is genuinely rare, the precision-recall curve is more informative than the ROC curve.

Q10.4 — What is oversampling? What is undersampling?

Oversampling: make more copies of the rare class so the model sees it more. Risk — it can memorise those copies.

Undersampling: throw away some of the common class. Risk — you're deleting real data.

Q10.5 — What is SMOTE?

Instead of duplicating rare examples, it invents new ones in between existing rare examples. So you get variety rather than copies.

Q10.6 — What's the biggest mistake people make with SMOTE?

Running it **before** splitting into train and test. Then synthetic points based on your test examples end up in training, your score looks wonderful, and it's meaningless.

Resample inside the training fold only. Always.

Q10.7 — What is `class_weight='balanced'`?

It tells the model that mistakes on the rare class cost more, so it stops ignoring them. One line of code, no new data invented.

Try this before SMOTE. It's simpler and it usually works.

Q10.8 — How does threshold tuning help?

The 0.5 cutoff is arbitrary — nothing special happens there. Moving it lets you catch more of the rare class at the cost of more false alarms, or the other way round.

You pick the point based on what each kind of mistake actually costs.

Q10.9 — For churn, which error is worse?

Missing a churner. That loses you the whole customer. A false alarm costs you one unnecessary discount offer.

So you tune for catching them, and you say so.

Q10.10 — How does imbalance show up in object detection?

Two ways. Between your classes — far fewer goggles than helmets in the footage. And within every single image, where the vast majority of the picture is background and only a small part is an object.

Q10.11 — What is focal loss?

A tweak that makes the model pay less attention to examples it's already getting easily right, so training focuses on the hard ones. Built specifically for the background-versus-object imbalance in detection.

Q10.12 — What is a confusion matrix, and how do you read one?

A small table showing four numbers: correctly caught, correctly ignored, false alarms, and misses.

Read it before any single summary number, because it tells you **which** mistake the model is making. Two models with identical accuracy can be making completely different errors.

The metrics

Q11.1 — Define precision and recall.

Precision: of everything I flagged, how much was actually right? **Recall:** of everything that was actually there, how much did I catch?

Q11.2 — How do you remember which is which?

Precision's bottom number is **your predictions**. Recall's bottom number is **reality**.

Or: precision is “when I shout, am I right?” Recall is “do I shout every time I should?”

Q11.3 — What is F1 score? Why the harmonic mean?

A single number combining precision and recall. It uses the harmonic mean rather than a plain average because that punishes being lopsided — you can't score well by maxing one and ignoring the other.

Q11.4 — When do you optimise precision over recall?

When false alarms are expensive. A spam filter that bins real emails is worse than one that lets a few spams through.

Optimise recall when **missing** things is expensive — disease screening, fire detection, churn.

Q11.5 — For your fire detector, which matters more?

Recall, clearly. A missed fire is catastrophic; a false alarm costs one wasted email. So you'd deliberately run a low confidence threshold and accept reviewing some false positives.

Q11.6 — What is accuracy, and when is it misleading?

How often you were right overall. It misleads whenever one outcome is much more common than the other — your 76% against a 73.5% baseline is the textbook example.

Q11.7 — What is specificity?

Of everything that genuinely wasn't there, how much did you correctly leave alone. It's recall, measured on the negative class.

Q11.8 — What is an ROC curve? What is AUC?

As you move the cutoff from strict to lenient, you catch more real cases but also raise more false alarms. The ROC curve plots that trade-off.

AUC is the area underneath it, which works out to: if you picked one real case and one non-case at random, how often does the model score the real one higher? 0.5 is coin-flipping; 1.0 is perfect.

Q11.9 — ROC-AUC vs PR-AUC?

When the thing you're looking for is rare, ROC-AUC flatters the model, because false alarms are being divided by an enormous number of negatives and so look tiny.

PR-AUC only looks at the rare class, so it tells the truth. Use it when positives are rare.

Q11.10 — What is log loss?

It scores not just whether you were right but how confident you were. Being confidently wrong is punished heavily. Use it when the actual probability matters, not just the yes/no.

Q11.11 — What regression metrics do you know?

Mean absolute error, mean squared error, root mean squared error, R-squared, and mean absolute percentage error.

Q11.12 — MAE vs RMSE — when do you pick which?

RMSE squares the errors first, so one enormous mistake hurts far more than several small ones. Use it when big misses are disproportionately bad.

MAE is in the natural units of your data and isn't thrown off by a single outlier.

Q11.13 — What is R-squared? Can it be negative?

The fraction of the variation your model explains. Yes, it can be negative — that means your model is doing worse than simply always predicting the average.

Q11.14 — What is adjusted R-squared?

R-squared with a penalty for the number of columns you used, so adding useless features doesn't inflate it.

Q11.15 — What is IoU?

How much a predicted box and the true box overlap, divided by the total area they cover between them. 1.0 is perfect overlap, 0 is none. It's how you decide whether a detection counts as correct.

Q11.16 — What is mAP, and what does mAP50-95 mean?

mAP is the overall detection score: for each class you measure how well it finds objects across all confidence levels, then average across classes.

mAP50 counts a box as correct if it overlaps by at least half. mAP50-95 repeats that at steadily stricter overlap requirements and averages — so it also grades how tightly your boxes fit.

Q11.17 — What is a precision-recall curve, and how do you use it?

It shows what precision you'd get at each level of recall as you move the cutoff. You use it to **pick the operating point**, not just to produce a number. That's the part people forget.

Q11.18 — What is model calibration?

Whether a predicted 0.8 really means that eight out of ten such cases turn out positive. Check it by bucketing predictions and comparing to what actually happened.

Random forests are often badly calibrated — they're reluctant to say 0.95 or 0.05 because they're averaging votes.

Q11.19 — What is a baseline, and why must you always report one?

The simplest possible predictor — always guess the most common answer. Without it, a metric is meaningless.

That's exactly the lesson of your 76% against 73.5%. A number with no baseline isn't a result.

Q11.20 — What are BLEU, ROUGE and perplexity used for?

BLEU for translation, ROUGE for summarisation — both compare generated text against a reference. Perplexity measures how surprised a language model is by text; lower is better.

Q11.21 — How do you evaluate a RAG system?

Split it in two and measure separately.

Did it find the right passage? Measure how often the correct passage appears in the top results.

Did it answer well from that passage? Measure whether the answer is actually supported by the retrieved text, and whether it made anything up.

Measuring only the final answer means you can't tell which half broke.

PRACTICE | The two to drill

Precision and recall. Say them out loud until “of what I flagged” and “of what was there” come instantly. Then do the mapping for all four of your projects: fire is recall, churn is recall, document QA is precision of the retrieval, PPE is both with per-class numbers.

CHAPTER RECAP

- Accuracy misleads whenever one class dominates — always quote the baseline.
- Precision is “when I flag it, am I right”; recall is “do I catch everything”.
- Fire and churn both need recall. Say so and say why.
- Resample inside the training fold only, or your score is fiction.
- Try `class_weight='balanced'` before SMOTE.
- Read the confusion matrix before any single number.
- The precision-recall curve is for choosing your cutoff, not just reporting.

PART IV

Deep Learning and Vision

Neural networks from the forward pass to the training loop, then the vision stack your strongest project is built on.

09 Deep Learning

10 CNNs, YOLO and OpenCV

Deep Learning

From the forward pass to the training loop

CORE

PREREQS: Chapter 7

You list PyTorch, so expect the whole chain: what a network is, what the loss does, how the gradients get back, and what the optimiser does with them. Then the practical PyTorch questions, which are where interviewers find out whether you have actually trained anything.

WHAT YOU WILL LEARN

- Explain a neural network and backpropagation without equations
- Say why activation functions exist in one sentence
- Answer the PyTorch traps: `zero_grad`, `eval mode`, `no_grad`
- Write a training loop from memory

What a network is

Q12.1 — What is a neural network, in one sentence?

Layers of simple calculations stacked up, where each layer's output feeds the next, and all the numbers inside get nudged repeatedly until the output is close to what you wanted.

Q12.2 — What is a perceptron?

A single unit: multiply each input by a weight, add them up, add a bias, push the result through a simple function. On its own it can only separate things that a straight line can separate.

Q12.3 — Why do we need activation functions at all?

Without them, stacking layers is pointless — a stack of straight-line operations is still just one straight line. The bend is what lets depth buy you anything.

Q12.4 — Compare sigmoid, tanh and ReLU.

Sigmoid squashes everything into 0 to 1, but at the extremes it flattens out and learning stops. Tanh is the same idea centred on zero instead, with the same flattening problem.

ReLU just sets negatives to zero and leaves positives alone. It's cheap, it doesn't flatten out for positive values, and it's the default for hidden layers.

Q12.5 — What is the dying ReLU problem, and the fix?

If a unit's input goes negative and stays negative, it outputs zero forever and never learns again — it's dead.

Fix: use a version that lets a small signal through for negatives instead of zero. LeakyReLU, ELU or GELU.

Q12.6 — What is softmax?

Turns a set of scores into probabilities that add up to 1. It's what you put on the output when you're picking one class out of several.

Q12.7 — When do you use sigmoid instead of softmax on the output?

When the classes aren't mutually exclusive — something can be several things at once.

Which is exactly your PPE case: one worker can be missing a helmet **and** gloves. Softmax would force them to compete; sigmoid lets each be judged independently.

Q12.8 — What is a loss function? Name the ones you'd use.

A single number saying how wrong the model currently is. Training is just making that number smaller.

Cross-entropy for classification. Mean squared error for predicting numbers. YOLO uses a combination — one part for the box position, one for the class.

Q12.9 — Explain forward and backward propagation.

Forward: the input flows through the layers and comes out as a prediction, which gets compared to the answer to give a loss.

Backward: that error gets traced back through the layers, working out how much each weight contributed to it, so each one knows which way to move.

Q12.10 — What is gradient descent? Batch vs stochastic vs mini-batch?

Nudge every weight slightly in the direction that reduces the error. Repeat.

Batch uses all your data for each nudge — accurate but slow. Stochastic uses one example — fast but jumpy. Mini-batch uses 32 to 256 and is what everybody actually does.

Q12.11 — What is the learning rate, and what happens if it's wrong?

How big each nudge is.

Too big and the model overshoots and the error bounces around or blows up. Too small and it takes forever, or gets stuck somewhere mediocre.

Q12.12 — What is a learning-rate schedule? Name two.

Reducing the step size as training goes on — big steps early to get roughly there, small steps later to settle in.

Step decay cuts it at fixed points. Cosine decay reduces it smoothly. Transformers usually also warm up at the start.

Q12.13 — Compare SGD, momentum, RMSprop and Adam.

SGD is the plain version. Momentum remembers which direction it's been going and keeps rolling that way, which helps it push through noise. RMSprop gives each weight its own step size based on how big its recent nudges were. Adam does both, and it's the default.

Q12.14 — What is the vanishing gradient problem?

As the error signal travels back through many layers, it gets multiplied by small numbers over and over until it's effectively zero — so the early layers stop learning entirely.

Fixed with ReLU, normalisation, and skip connections that give the signal a shortcut back.

Q12.15 — What is exploding gradient, and the fix?

The opposite — the signal gets multiplied up until the weights blow up and you get NaN. Fix by capping the size of the update. One line of code.

Q12.16 — What is batch normalisation, and why does it help?

It rescales each layer's inputs so they stay in a sensible range instead of drifting as training progresses. Training becomes faster and much more stable.

Q12.17 — Batch norm vs layer norm — which does a transformer use?

Batch norm normalises across all the examples in the batch. Layer norm normalises within one example, across its features.

Transformers use layer norm, because sentence lengths and batch sizes vary and batch norm doesn't cope well with that.

Q12.18 — What is dropout? Does it run at inference?

During training, randomly switch off a fraction of the units each step. That stops the network leaning on any single one and forces it to spread the work.

It's switched off at prediction time — which is exactly what `model.eval()` does.

Q12.19 — What is early stopping?

Stop training when the validation score stops improving, and keep the best version you saw. It's the cheapest way to prevent overfitting there is.

Q12.20 — What is weight initialisation, and why not use zeros?

Starting all weights at zero means every unit in a layer computes exactly the same thing and receives exactly the same update — so they never become different from each other and the layer is useless.

So you start with small random values, scaled to suit your activation function.

Q12.21 — Epoch vs batch vs iteration?

An epoch is one full pass over all your data. A batch is one group of examples. An iteration is one weight update, which is one batch.

Q12.22 — How do you pick batch size?

The biggest that fits in your GPU memory and still trains well. Bigger is faster per epoch but sometimes generalises slightly worse, and may need a higher learning rate to compensate.

PyTorch in practice**Q12.23 — What is transfer learning? Why does it matter for all your projects?**

Start from a model already trained on a huge dataset, then adapt it to yours.

It's the entire reason you could train a fire detector or a five-class PPE model on a modest custom dataset. The pretrained model already knows what edges, textures and object shapes look like — you're only teaching it your specific objects.

Q12.24 — Fine-tuning vs feature extraction?

Feature extraction: freeze the pretrained part completely and train only the new output layer. Fine-tuning: unfreeze some or all of it and train gently at a low learning rate.

Freeze more when you have less data.

Q12.25 — What is data augmentation? Which are safe for PPE detection?

Making extra training examples by altering the ones you have — flipping, rotating, changing brightness.

Safe for your case: horizontal flip, scaling, brightness and contrast, slight rotation.

Risky: flipping vertically, because workers aren't upside down. And aggressive colour shifts, because a high-vis vest is **defined** by its colour — shift it and you've destroyed the signal.

Q12.26 — What is a hyperparameter you tuned, and how?

Have a real one ready. Input resolution, confidence threshold, learning rate or number of epochs — tuned by watching the validation score. Name the number you watched.

Q12.27 — Tensor vs NumPy array?

A tensor can live on a GPU, and it remembers the operations done to it so gradients can be computed. Otherwise the two are deliberately very similar.

Q12.28 — What is autograd in PyTorch?

It quietly records every operation you do, building a map of how the result was computed. Then `loss.backward()` walks that map backwards and works out every gradient for you.

Q12.29 — What does `optimizer.zero_grad()` do, and what breaks without it?

PyTorch **adds** new gradients to the old ones rather than replacing them. So if you don't clear them each step, you're updating with the sum of every batch so far and the model trains wrongly.

Classic interview trap. Know this one.

Q12.30 — Write a minimal PyTorch training loop.

```

1  for xb, yb in loader:
2      optimizer.zero_grad()
3      out = model(xb)
4      loss = criterion(out, yb)
5      loss.backward()
6      optimizer.step()

```

Be able to write this from memory. Five lines: clear, predict, score, blame, update.

Q12.31 — `model.train()` vs `model.eval()`?

They switch dropout and batch norm between their training and prediction behaviour. Forgetting `eval()` when predicting means dropout is still randomly switching units off, so you get different answers each time you run it.

Q12.32 — What does `torch.no_grad()` do?

Tells PyTorch to stop recording operations, because you're not going to train on this. Less memory, faster. Wrap your whole prediction path in it.

Q12.33 — PyTorch vs TensorFlow?

PyTorch feels like ordinary Python and you can step through it with a debugger, which is why research moved to it. TensorFlow has a longer history on the deployment side.

Say you use PyTorch and can read Keras. That's the honest answer.

Q12.34 — What is a Dataset and a DataLoader?

A Dataset knows how to fetch one example by number. A DataLoader wraps it and handles batching, shuffling, and loading in parallel while the GPU works.

Q12.35 — How do you save and load a model?

Save the weights (`state_dict`), not the whole Python object. Then build the same architecture and load the weights into it.

Saving the whole object ties the file to your exact code layout, and it breaks the moment you move a file.

Q12.36 — What is mixed-precision training?

Doing most of the arithmetic in half precision while keeping a full-precision master copy of the weights. Roughly halves memory and speeds things up, with almost no accuracy cost on modern GPUs.

Q12.37 — Why do we train on GPUs?

Because training is thousands of independent multiplications happening at once, and a GPU has thousands of small cores to do them simultaneously. A CPU has a handful of fast ones, which is the wrong shape for this job.

Q12.38 — Your model trains fine but inference is slow. What do you do?

Measure first, to find out whether it's the model or the preprocessing around it.

Then: batch the requests, make sure you're in eval mode with gradients off, export to a faster runtime, reduce the number precision, lower the input resolution, or use a smaller model variant.

PRACTICE | The three traps

`zero_grad`, `model.eval()` and `no_grad()`. All three are asked constantly, all three are one-sentence answers, and all three catch people who have copied a training loop without understanding it.

CHAPTER RECAP

- Activation functions are what make depth worth anything.
- Sigmoid on the output for PPE, because a worker can be missing two things at once.
- Backprop traces the error back to find out which weight is responsible.
- PyTorch adds gradients rather than replacing them — hence `zero_grad`.
- Transfer learning is why your small datasets worked at all.
- Don't flip PPE footage vertically, and don't shift the colours — the vest is defined by its colour.

CNNs, YOLO and OpenCV

Your strongest area, and the one they will push hardest

CORE

PREREQS: Chapter 9

This is where your best project lives, so expect the longest and most detailed questioning here. Know how YOLO works, not just how to call it — the difference shows immediately. The video-splitting question in this chapter is the one a sharp interviewer will use to find out whether you really understand your own evaluation.

WHAT YOU WILL LEARN

- Explain convolution, pooling and receptive field in plain words
- Describe how YOLO works and why you chose it
- Answer the v5-versus-v8 question properly
- Know the one video-splitting mistake that invalidates a score

Convolutional networks

Q13.1 — Why a CNN instead of a fully connected net for images?

Two reasons. A fully connected layer on a 640×640 image needs an enormous number of weights. And it would have to learn separately that a helmet in the top-left and a helmet in the bottom-right are both helmets.

A convolution slides the same small detector across the whole image, so it learns “helmet” once and finds it anywhere, with far fewer weights.

Q13.2 — What is a convolution operation?

A small window slides across the image, and at each position it multiplies what it sees by a set of learned numbers and adds them up. The result is a map showing where that particular pattern appeared.

Q13.3 — What is a filter, and what does it learn?

The set of numbers in that window. Early layers learn simple things — edges, colour changes. Middle layers combine those into textures and shapes. Deep layers combine those into whole objects.

Q13.4 — What are stride and padding?

Stride is how far the window jumps each step. A bigger stride means a smaller output. Padding adds a border of zeros around the edge so the output can stay the same size as the input.

Q13.5 — Give the output-size formula.

```
out = floor((in - kernel + 2×padding) / stride) + 1.
```

Expect to compute this on a whiteboard. Practise it once with real numbers.

Q13.6 — What is pooling? Max vs average?

Shrinking the map by summarising each small patch into one number.

Max pooling keeps the strongest response, which is what you want when asking “is this thing present anywhere here?” Average pooling smooths. Both make the network slightly less fussy about exact position.

Q13.7 — What is a receptive field?

How much of the original image one unit deep in the network can actually “see”. It starts tiny and grows with depth — which is how deep layers end up recognising whole objects rather than edges.

Q13.8 — What is a 1×1 convolution for?

Mixing and reducing the number of channels cheaply, without looking at neighbouring pixels at all. Used everywhere to keep models small.

Q13.9 — What is a residual or skip connection?

Adding a layer’s input to its output, so the layer only has to learn the **difference** rather than the whole thing.

It also gives the error signal a shortcut straight back during training, which is what made very deep networks trainable at all.

Q13.10 — Why are deeper networks hard to train?

The error signal fades as it travels back, so early layers barely learn. And counterintuitively, adding layers can make accuracy worse. Skip connections and normalisation are what fixed both.

Q13.11 — What is global average pooling, and why use it over flatten?

Instead of unrolling the whole feature map into a huge list, you just average each channel down to one number. Far fewer weights, and it works with different input sizes.

Q13.12 — Name some classic CNN architectures and what each contributed.

LeNet proved the idea. AlexNet added ReLU and GPUs. VGG showed that stacking simple 3×3 layers works. Inception used several filter sizes in parallel. ResNet introduced skip connections. MobileNet made them small enough for phones.

Q13.13 — What is a depthwise separable convolution?

Splitting the work in two: first look at each channel's spatial pattern separately, then mix the channels. Several times less computation for nearly the same result. It's the basis of models that run on edge devices.

Q13.14 — How does a CNN handle different input image sizes?

Usually it doesn't — you resize or pad everything to one size first. If the network ends with global pooling rather than a fixed dense layer, it can tolerate some variation.

Q13.15 — What is a feature map?

The output of one filter applied across the whole image — a map showing where that pattern was found.

Q13.16 — What are Vision Transformers, and when would you use one?

They chop the image into patches and let every patch look at every other patch, instead of sliding a window.

They beat CNNs when you have enormous amounts of data. With limited data, and for real-time detection, CNNs are still the right choice — which is the honest answer for your projects.

YOLO

Q14.1 — Classification vs detection vs segmentation?

Classification labels the whole picture: "there is a helmet in this image". Detection adds boxes: "there is a helmet, **here**". Segmentation labels every single pixel, so you get the exact outline.

Q14.2 — Why detection rather than classification for SafeSight?

Because the question is about individual workers, not the image. You need to know **which** person is missing **which** item, and where they are, to produce evidence a supervisor can act on.

Q14.3 — What does YOLO stand for, and what's the core idea?

"You Only Look Once." Older detectors first proposed regions that might contain something, then classified each one — two passes. YOLO does the whole thing in one pass over the image, which is what makes it fast enough for live video.

Q14.4 — One-stage vs two-stage detectors?

Two-stage proposes candidate regions then examines each — historically more accurate, much slower. One-stage predicts everything directly in a single pass — fast enough for video, which is why you used it.

Q14.5 — Why YOLO rather than Faster R-CNN for CCTV?

Throughput. You have multiple camera streams running continuously; the two-stage model couldn't keep up. And modern YOLO has closed most of the accuracy gap anyway.

Q14.6 — What are anchor boxes?

A set of preset box shapes at each position. The model doesn't predict a box from nothing — it predicts how to adjust the nearest preset shape, which is an easier job.

Q14.7 — What is anchor-free detection, and which of your models uses it?

Predicting the box centre and size directly, with no preset shapes to adjust.

YOLOv8 is anchor-free; YOLOv5 is anchor-based. That's a genuine difference between your two projects and a likely question.

Q14.8 — Main differences between YOLOv5 and YOLOv8?

v8 drops anchor boxes, splits the classification and box-position jobs into separate heads instead of one shared head, removes the separate “is there anything here” score, and ships with segmentation and pose as well as detection.

Both are from Ultralytics and feel the same to use, which is why it's easy to mistake it for a version bump.

Q14.9 — What do the n/s/m/l/x suffixes mean?

Model size — how wide and deep it is. Bigger means more accurate and slower. You used the medium one.

Q14.10 — What is non-maximum suppression?

After detection you usually have several overlapping boxes on the same object. NMS keeps the most confident one and deletes anything overlapping it heavily, then repeats.

Q14.11 — What if the NMS threshold is too high or too low?

Too high and you keep several boxes on one person. Too low and you merge two people standing close together into one detection — which is a real risk on a busy work site.

Q14.12 — What is the confidence threshold, and how did you choose yours?

The minimum score before you report a detection. Lower it to catch more and accept false alarms; raise it to be more certain and miss more.

For a safety system you lean towards catching more, because a missed violation is worse than a supervisor glancing at a wrong one. For fire, even more so.

Q14.13 — What is YOLO's loss made of?

Three parts: how far off the box position was, whether the class was right, and — in v5 — whether there was anything there at all. v8 drops that third part.

Q14.14 — What is mosaic augmentation?

Stitching four training images into one, so each training example contains more objects, at more sizes, in more contexts. Ultralytics turns it on by default and turns it off for the last few epochs.

Q14.15 — What is letterboxing, and why not just resize?

Scaling the image to fit and padding the leftover space with grey, instead of stretching it.

A plain resize squashes people into the wrong proportions, which makes them harder to detect.

Q14.16 — How do you label data for detection? What format does YOLO want?

Draw a box round each object in a tool like Roboflow, LabelImg or CVAT.

YOLO wants one text file per image, one line per object: class, centre-x, centre-y, width, height — all as fractions of the image size.

Q14.17 — What is in a `data.yaml`?

Where the train, validation and test images are, how many classes there are, and their names. Your fire config is one class, named “fire”. Know that.

Q14.18 — How do you split video data for training? What’s the trap?

Split by clip or by camera — never by random frame.

This is the important one. Frames half a second apart are almost identical. If you split randomly, nearly identical pictures end up in both training and validation, so the model is being tested on things it has effectively already seen. Your score comes out far too high and you find out in production.

A sharp interviewer will ask this specifically.

Q14.19 — Your 5 PPE classes are imbalanced. How do you handle that?

Collect more pictures of the rare items. Oversample the images that contain them. Use copy-paste augmentation to put more of them into scenes. Raise the input resolution so small items are bigger.

And report the score per class rather than hiding behind the average.

Q14.20 — How do you detect small objects better?

Higher input resolution. Run detection on tiles of the image rather than the whole thing shrunk down. Use the earlier, higher-resolution layers of the network. Match the preset box sizes to the actual object sizes.

Q14.21 — What is an FPN or PANet?

A way of combining the network’s detailed early layers with its meaning-rich deep layers, so the detector works on both small and large objects. YOLO’s middle section does this.

Q14.22 — How do you link a PPE violation to a specific worker?

Detect people and detect safety items separately, then work out which item belongs to which person — usually by checking whether the item’s box sits inside or overlaps the person’s box. Then track people across frames so the identity stays stable.

This is the real engineering problem in SafeSight. Be ready to say exactly which approach you used and where it still fails — overlapping workers is the honest weak point.

Q14.23 — What is object tracking, and name a tracker?

Keeping the same identity on the same person as they move between frames, so one worker isn’t counted as a new person every frame. SORT, DeepSORT, ByteTrack, BoT-SORT.

Q14.24 — What's the difference between detection and tracking in your event log?

Enormous. Without tracking, a worker standing without a helmet for thirty seconds generates hundreds of separate events. With tracking, that becomes **one** violation with a duration.

That's the difference between a usable audit trail and noise.

Q14.25 — What is FPS, and what determines it in your pipeline?

Frames processed per second. Determined by model size, input resolution, whether you're on a GPU, video decoding cost, and the NMS step.

You can also just skip frames. PPE compliance doesn't need thirty frames a second — five is plenty, and that multiplies how many cameras you can handle.

Q14.26 — How do you deploy a detector on edge hardware?

Export it to a faster runtime, reduce the number precision, pick a smaller model variant, and lower the resolution. Jetson devices if you need a GPU at the edge.

Q14.27 — What is ONNX?

A common file format for models, so one trained in PyTorch can run in a different, faster runtime without rewriting it.

Q14.28 — What is quantisation? What does it cost?

Storing the model's numbers at lower precision — 8-bit instead of 32-bit. Smaller and faster, usually with only a small accuracy loss.

But measure it, don't assume it. On small objects the loss can be bigger than you expect.

Q14.29 — What is model pruning?

Removing the weights or channels that barely contribute, making the model smaller. Usually followed by a bit more training to recover what you lost.

Q14.30 — What is knowledge distillation?

Training a small model to copy a large model's outputs. You keep most of the accuracy at a fraction of the cost.

Q14.31 — How would you handle night footage or a dirty lens?

Mainly by training on it — include low-light and degraded footage in the dataset. Beyond that: infrared cameras, contrast enhancement, and monitoring for when a camera's picture quality shifts so you know it needs cleaning.

Q14.32 — How do you know your deployed detector still works next month?

You don't have labels in production, so you watch for changes instead. Track how many detections each camera produces and how confident they are. A sudden drop on one camera means something changed — the lighting, the lens, or the scene.

Then sample some frames for a human to check.

OpenCV

Q15.1 — What is OpenCV?

A library for the practical side of working with images and video — reading files, grabbing webcam frames, resizing, cropping, drawing boxes, writing video out.

Q15.2 — How do you read and display an image?

`cv2.imread(path)`, then `cv2.imshow` and `cv2.waitKey(0)`.

Q15.3 — What's the gotcha with OpenCV colour channels?

OpenCV reads images as Blue-Green-Red. Almost everything else uses Red-Green-Blue.

Forget to convert, and your model sees an image with the red and blue swapped. It still runs — it just quietly performs worse. Very common bug.

Q15.4 — How do you capture webcam frames?

Open the camera, then loop reading one frame at a time, and release it at the end. Your fire detector does exactly this.

Q15.5 — Why does your inference loop stutter if you send email inline?

Because sending an email takes a couple of seconds, and during that time nothing is reading frames. The camera buffer fills up and you start dropping frames or processing old ones.

Moving the email to a separate thread is why your fire detector stays real-time. This is your best story in this section — it's specific, you diagnosed it, and the fix is clean.

Q15.6 — How do you resize and crop?

`cv2.resize(img, (w, h))` to resize. To crop, just slice the array: `img[y1:y2, x1:x2]`.

Q15.7 — How do you convert to greyscale, and why bother?

`cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)`. It's a third of the data, and plenty of classical operations only care about brightness, not colour.

Q15.8 — What is Gaussian blur for?

Smoothing out noise before you look for edges or apply a threshold. Without it, every speck of noise looks like an edge.

Q15.9 — What is thresholding? What is Otsu's method?

Turning a greyscale image into pure black and white at some cutoff. Otsu's method picks that cutoff for you automatically based on the image.

Q15.10 — What is Canny edge detection?

A multi-step edge finder: blur, find where brightness changes sharply, thin those down to single lines, then keep the strong ones and any weak ones connected to them.

Q15.11 — What are morphological operations?

Simple shape operations on black-and-white images. Erosion shrinks the white regions, dilation grows them. Combining them removes small specks or fills small holes. Used to tidy up masks.

Q15.12 — What is contour detection used for?

Finding the outlines of connected shapes in a black-and-white image, so you can get each one's area, centre or bounding box.

Q15.13 — How do you draw boxes and labels on a frame?

`cv2.rectangle` and `cv2.putText`. That's how your evidence images get their annotations.

Q15.14 — How do you write an output video?

Create a `VideoWriter` with the output path, codec, frame rate and frame size, then write each frame to it.

Q15.15 — Classical CV vs deep learning — is OpenCV obsolete?

No. Deep models do the recognition; OpenCV does everything around them — reading the video, resizing, colour conversion, drawing the output, saving the result. Every one of your vision projects uses both.

PRACTICE | The two that decide this round

Q14.18 — splitting video by clip rather than frame. And Q14.22 — how you link a violation to a specific worker. Those two separate someone who ran a YOLO tutorial from someone who built a system.

CHAPTER RECAP

- A convolution slides one detector across the whole image, which is why it needs far fewer weights than a dense layer.
- YOLO does detection in one pass, which is what makes live video possible.
- v8 is anchor-free, v5 is anchor-based — know this, it's your two projects.
- Split video by clip, never by random frame. This is the question that catches people.
- Tracking turns hundreds of frame events into one violation with a duration.
- OpenCV reads images as BGR, not RGB.
- The threaded email fix is your best concrete engineering story.

PART V

Language Models

How a transformer works, how retrieval works, and how to talk about agents without overclaiming. Two of your projects live here.

-
- 11 NLP and Transformers
 - 12 Embeddings, Vector Search and RAG
 - 13 Prompting, LangChain and Agents

NLP and Transformers

How a language model actually works

CORE

PREREQS: Chapter 9

You list LLMs, Llama 3 and Ollama, so they will ask how a transformer works. The attention answer is the one that matters — get it right in plain words and the rest of the section goes easily. The older NLP material underneath it still gets asked because it is the floor your RAG projects stand on.

WHAT YOU WILL LEARN

- Explain attention without writing the formula
- Say what query, key and value mean in ordinary words
- Describe how an LLM is trained and what fine-tuning changes
- Answer the hallucination question honestly

Classic NLP

Q16.1 — What is tokenisation?

Chopping text into the pieces a model actually reads. Sometimes whole words, usually chunks of words.

Q16.2 — What is subword tokenisation, and why do LLMs use it?

Common words stay whole; rare words get broken into familiar pieces. “Unhappiness” might become “un”, “happi”, “ness”.

Why: it keeps the vocabulary small, and the model never meets a word it can’t represent at all — worst case it spells it out in pieces.

Q16.3 — What are stemming and lemmatisation?

Both cut words back to a root so “running” and “runs” count as the same thing.

Stemming chops crudely and can produce non-words — “studies” becomes “studi”. Lemmatisation uses a dictionary and gives you a real word — “studies” becomes “study”.

Q16.4 — What are stop words? Should you always remove them?

Very common words like “the”, “is”, “and”.

Remove them for simple word-counting models, where they're just noise. Keep them for transformers, where the grammar genuinely carries meaning.

Q16.5 — What is bag of words?

Representing a document by which words appear and how often, completely ignoring the order. Crude, but surprisingly effective for classification.

Q16.6 — What is TF-IDF?

A way of scoring words that rewards being frequent in **this** document but rare across all documents. So “the” scores near zero and “defibrillator” scores high.

Q16.7 — What are n-grams?

Short runs of consecutive words. They recover a little of the word order that bag-of-words throws away — “not good” stays distinguishable from “good”.

Q16.8 — What are word embeddings? Name two methods.

Turning each word into a list of numbers, arranged so that words with similar meanings end up close together. Word2Vec and GloVe.

Q16.9 — Word2Vec: CBOW vs skip-gram?

CBOW guesses a word from the words around it. Skip-gram does the reverse — guesses the surroundings from the word. Skip-gram handles rare words better.

Q16.10 — What's the limitation of Word2Vec that transformers solved?

One fixed vector per word, forever. So “bank” has a single meaning, whether you mean money or a river.

Transformers produce a different vector depending on the sentence, so “bank” means different things in different contexts.

Q16.11 — What is named entity recognition?

Picking out and labelling the people, companies, places and dates in text. Useful in document question-answering for pulling structured facts out.

Q16.12 — What is POS tagging?

Labelling each word with its part of speech — noun, verb, adjective.

Q16.13 — What is an RNN, and what's its problem?

It reads a sequence one item at a time, carrying a running memory.

Two problems: it has to go in order, so it can't be parallelised and is slow. And the memory fades, so by the end of a long sentence it has forgotten the beginning.

Q16.14 — How do LSTM and GRU help?

They add gates that decide what to keep, what to forget and what to pass on. That lets information survive much longer. GRU is a lighter version with fewer gates.

Q16.15 — What is seq2seq, and why was attention added?

One network reads the input and squeezes it into a single summary; another reads that summary and produces the output.

Attention was added because a single fixed summary can't hold a long sentence. Instead of one summary, the output side gets to look back at **every** part of the input and decide what's relevant right now.

Q16.16 — Text classification vs sentiment vs NER — in terms of output shape?

Classification and sentiment: one label for the whole document. NER: one label per word.

Knowing the output shape is how you decide what to put on top of the model.

Q16.17 — What is cosine similarity, and why use it over distance for text?

It measures the angle between two vectors, ignoring how long they are.

For text that matters: a long document and a short one about the same topic should count as similar. Using straight-line distance would say they're far apart just because one has more words. It's what your FAISS search uses.

Transformers

Q17.1 — What is a transformer?

A design where every word in a sentence can look directly at every other word, all at once, instead of reading left to right.

That's what makes it fast to train — the whole sentence is processed in parallel rather than one word at a time.

Q17.2 — What is self-attention, in plain terms?

Each word looks at every other word in the sentence and decides how much each one matters to it. Then it builds a new version of itself that's blended from the words it cared about.

So in “the cat sat on the mat because it was tired”, the word “it” can look back, notice “cat” is the relevant word, and take meaning from it.

Q17.3 — Explain query, key and value.

Think of it as a search.

Each word puts out a **query** — “here's what I'm looking for”. Each word also offers a **key** — “here's what I am”. The query is compared against every key to see which match. Then each word contributes its **value** — “here's what I actually give you” — weighted by how well it matched.

Q17.4 — Give the attention formula.

Compare every query against every key, scale the result down, turn it into percentages with softmax, and use those percentages to blend the values.

Written out: $\text{softmax}(\mathbf{QK}^T / \sqrt{d}) \mathbf{V}$. The division stops the numbers getting so large that softmax picks one word and ignores everything else.

Q17.5 — What is multi-head attention for?

Running several attention calculations side by side, so different ones can track different relationships — one might follow grammar, another might track which pronoun refers to which noun. Then the results get combined.

Q17.6 — Why do transformers need positional encoding?

Because attention has no sense of order — to it, a sentence is just a bag of words all looking at each other. So you have to explicitly add information about where each word sits.

Modern models like Llama use rotary encoding, which handles relative distance particularly well.

Q17.7 — Encoder-only vs decoder-only vs encoder-decoder?

Encoder-only reads and understands — BERT, and the embedding models you use for search. Decoder-only generates text — GPT, Llama. Encoder-decoder reads one thing and writes another — translation, the T5 family.

Q17.8 — What is causal or masked attention?

In a model that generates text, each word is only allowed to look **backwards** at earlier words. Otherwise it could see the answer it's supposed to be predicting, and it would learn nothing.

Q17.9 — What is the feed-forward layer doing in a transformer block?

Attention moves information **between** words. The feed-forward layer then processes each word on its own.

It's also where most of the model's parameters are, which is roughly where its stored knowledge lives.

Q17.10 — How is an LLM pretrained?

By covering up the next word and asking it to guess, over an enormous amount of text. No human labelling needed — the text is its own answer key.

Q17.11 — Difference between pretraining, fine-tuning and RLHF?

Pretraining teaches it language in general, from scratch, at huge scale. Fine-tuning adapts that general model to your specific task or style. RLHF nudges it towards answers people actually prefer, using human comparisons.

Q17.12 — What is instruction tuning?

Training on pairs of “here's a request” and “here's a good response”, so the model follows instructions instead of just continuing your text.

It's the difference between a base model and a chat model.

Q17.13 — What is a context window, and why does it constrain your RAG design?

The maximum amount of text the model can hold at once.

It's exactly why you chunk documents and only retrieve the best few passages rather than pasting in a whole PDF. The PDF wouldn't fit, and even if it did you'd be paying for all of it.

Q17.14 — Why is attention expensive for long contexts?

Because every word looks at every other word. Double the length and you’ve quadrupled the work. That’s what all the “efficient attention” research exists to fix.

Q17.15 — What is temperature? Top-k and top-p?

Temperature controls how adventurous the model is. Near zero it always picks the most likely next word. Higher and it takes more risks.

Top-k limits it to the k most likely words. Top-p limits it to the shortest list of words that covers a given share of the probability.

Q17.16 — What temperature for document QA, and why?

Near zero. In DocuMind you want it to faithfully report what the document says. Creativity is a defect here, not a feature.

Q17.17 — What is a hallucination? Why do they happen?

A confident, fluent, completely wrong answer.

They happen because the model is trained to produce text that **sounds** right, not text that **is** right. It has no internal sense of “I don’t actually know this.”

Q17.18 — How do you reduce hallucination?

Give it the source material and tell it to answer only from that. Make it cite which passage it used. Explicitly tell it to say “not found” rather than guess. Set temperature near zero. Then check the citations actually resolve.

Your citation design in DocuMind is exactly this.

Q17.19 — What is LoRA? What is QLoRA?

Normally fine-tuning means updating billions of weights. LoRA freezes them all and trains a tiny set of extra ones alongside — a fraction of a percent of the model, with nearly the same effect.

QLoRA does the same but also stores the frozen model in a compressed form, so a large model fits on one consumer GPU.

Q17.20 — What is PEFT?

The general name for this family — fine-tuning by changing a small number of parameters rather than all of them. LoRA is the best-known member.

Q17.21 — When do you fine-tune vs use RAG vs just prompt?

Prompt first, always — it’s free and takes minutes.

RAG when the problem is **missing or changing knowledge** — the model doesn’t know your documents.

Fine-tune when the problem is **behaviour** — it knows the material but won’t produce the format or tone you need consistently.

They solve different problems, and you can use both together.

Q17.22 — What is quantisation for LLMs? What does Ollama use?

Storing the model's numbers at lower precision so it fits on smaller hardware. Ollama serves models in a compressed format — which is how you ran Llama 3 on your own machine.

Q17.23 — What is Ollama, and why did you choose it?

A tool that downloads and runs language models locally, behind a simple API.

Why you chose it: no per-token bill, and no document ever leaves the machine. Lead with the privacy argument — for confidential documents it's the strong one.

Q17.24 — Open-source vs API models — how do you choose?

Open models when you need privacy, predictable cost, or offline operation. Hosted APIs when you need the strongest possible model and don't want to run anything.

For documents containing confidential material, local wins.

Q17.25 — What is a token, roughly how many words?

A chunk of a word. In English roughly three-quarters of a word on average, so a thousand tokens is about seven hundred and fifty words.

Q17.26 — What is KV caching?

When generating text, the model would otherwise redo the work for every previous word each time it adds a new one. Caching stores that work so each new word only costs the new work.

It's the main reason generation speeds up after the first word.

Q17.27 — What is a mixture of experts?

Instead of one big network, you have many smaller specialist networks and a router that picks a couple for each word.

So the model can hold far more knowledge without every word costing more to compute.

Q17.28 — What is chain-of-thought prompting?

Asking the model to work through its reasoning step by step before answering. It measurably improves anything with multiple steps.

Q17.29 — What are the risks of putting an LLM in a safety product?

Four. It can confidently cite a rule that doesn't exist. Instructions hidden in an ingested document can hijack it. The model can be updated underneath you and change behaviour silently. And operators can start trusting it more than they should.

For SafeSight the honest answer is that the LLM **explains and cites** — a human still signs off on anything being enforced. Saying that unprompted is a strong signal.

PRACTICE | The answer that carries this chapter

Q17.2, self-attention. Practise the “the cat... because it was tired” example out loud. If you can explain attention with a sentence rather than a formula, every other question in this chapter becomes easy.

CHAPTER RECAP

- Attention means every word looks at every other word and decides what matters.
- Query is what I'm looking for, key is what I am, value is what I give you.
- Positional encoding exists because attention has no sense of order.
- Hallucinations happen because the model optimises for sounding right, not being right.
- Prompt first, RAG for missing knowledge, fine-tune for stubborn behaviour.
- Ollama's selling point for DocuMind is privacy — lead with that.
- For a safety product, say plainly that the LLM assists and a human decides.

Embeddings, Vector Search and RAG

The section most likely to decide the interview

CORE

PREREQS: Chapter 11

Two of your projects are retrieval systems and FAISS appears twice on your resume, so expect detail here. The questions that matter most are how you debug a bad retrieval and how you would evaluate the thing — both are where candidates run out of answers.

WHAT YOU WILL LEARN

- Explain embeddings and vector search in plain words
- Say what FAISS is and, honestly, what it is not
- Walk through your RAG pipeline in thirty seconds
- Debug a wrong retrieval by working backwards through the stages

Embeddings and vector search

Q18.1 — What is an embedding?

Turning a piece of text into a list of numbers, arranged so that things with similar meanings end up near each other. “Hard hat” and “safety helmet” land close together even though they share no words.

Q18.2 — What is a vector database?

A store built for the question “what’s most **similar** to this?” rather than “find me the row with this exact ID”. Normal databases are terrible at the first question.

Q18.3 — What is FAISS, and is it a database?

A library for searching quickly through millions of these number-lists.

Strictly, it’s not a database — it’s an index. It doesn’t save itself to disk automatically, it has no way to filter by “only documents from 2024”, and it has no concept of users. You build all of that yourself.

Saying this shows you understand the tool’s limits, which is a better answer than praising it.

Q18.4 — Which embedding model did you use, and what are its properties?

MiniLM on SafeSight — 384 numbers per piece of text, small and fast, good general-purpose sentence matching.

Know the dimension count. It gets asked.

Q18.5 — How do you choose an embedding model?

Match it to your domain, check it on your own data rather than trusting a leaderboard, and weigh the number of dimensions against speed and storage.

And check its maximum input length is longer than your chunks — otherwise it's silently cutting the end off every chunk and you'd never know.

Q18.6 — What distance metric do you use, and why cosine?

Cosine similarity, which compares the direction of two vectors and ignores their length.

Because a long passage and a short one about the same thing should count as similar. If you normalise the vectors first — which you should — a simple dot product gives you the same answer faster.

Q18.7 — Exact vs approximate nearest neighbour search?

Exact compares your query against every single stored vector. Perfect results, but the time grows with the collection.

Approximate uses a clever structure to check only a promising subset. You give up a tiny bit of accuracy for an enormous speedup. You need it past roughly a million items.

Q18.8 — Name some FAISS index types and when to use each.

Flat: compare against everything. Perfect, fine up to a few hundred thousand.

IVF: group the vectors into clusters first, then only search the nearest clusters.

HNSW: build a navigable graph and walk it towards the answer. Fast and accurate, uses more memory. The usual default now.

PQ: compress the vectors so the index fits in less memory, at some accuracy cost.

Q18.9 — What is HNSW?

A layered graph where you start at a sparse top layer, hop towards roughly the right area, then drop into denser layers for precision. Like zooming in on a map.

Q18.10 — What is IVF's `nprobe`, and what does tuning it do?

How many clusters to actually search. Search more and you find more of the right answers but it takes longer. It's the main accuracy-versus-speed dial.

Q18.11 — Why does an IVF index need training when a flat index doesn't?

Because it has to learn where the cluster centres are, which it does from a sample of your vectors. A flat index just stores everything, so there's nothing to learn.

Q18.12 — FAISS vs Chroma vs Pinecone vs pgvector — how would you choose?

FAISS when you want something local with no extra service to run. Chroma for a light local store that also handles metadata. Pinecone or Weaviate when you want someone else to run it at scale with filtering. pgvector when your data is already in Postgres and you'd rather have one system than two.

For your projects FAISS was the right call. Justify it on simplicity and cost.

Q18.13 — How do you persist and reload a FAISS index?

Write it out and read it back with FAISS's own functions — but you must save the actual chunk texts alongside it yourself.

The index only stores the numbers and an ID. Lose the sidecar file and you have an index that can tell you “result number 4,712 is the best match” and nothing that can tell you what 4,712 says.

Q18.14 — How do you handle updates and deletions?

Some index types support removing items; others don't really, so you rebuild the whole thing.

If your documents change often, that's the strongest argument for using a real vector database instead.

Q18.15 — What is metadata filtering, and what's the catch with FAISS?

Restricting a search — only this department, only this year.

FAISS can't do it. So you either keep separate indexes per category, or you fetch more results than you need and filter afterwards — which sometimes leaves you short.

Q18.16 — What is k in top- k retrieval, and how do you pick it?

How many passages you hand to the model.

Too few and you miss the answer. Too many and the right passage gets buried in irrelevant text, and you pay for all of it. Three to five is a sensible starting point; tune it on real questions.

Q18.17 — What is a reranker, and why add one?

A second, slower model that reads the question and each candidate passage **together** and rescores them.

The pattern: fetch fifty cheaply, rerank down to the best five. It's the single biggest easy improvement available in RAG — worth more than most other changes combined.

Q18.18 — Bi-encoder vs cross-encoder?

A bi-encoder turns the question and the document into numbers separately, so you can prepare the documents in advance. Fast, less accurate.

A cross-encoder reads both together, so it understands how they relate. Much more accurate, far too slow to run over a whole collection — which is why you only use it on the shortlist.

Q18.19 — What is hybrid search?

Combining meaning-based search with plain keyword search, then merging the rankings.

It fixes the one thing embeddings are bad at: exact strings. Part numbers, regulation codes, specific names. Very relevant to safety manuals, where “Reg 14(2)(b)” needs to match exactly.

RAG

Q19.1 — What is RAG, in one sentence?

Find the relevant passages from your own documents, paste them into the prompt, and let the model answer from those — so the answer comes from your data rather than the model’s memory.

Q19.2 — Why use RAG instead of fine-tuning?

Your documents change, and retraining takes hours every time. With RAG you add a document in seconds.

It also lets you cite the source, which fine-tuning can’t. And fine-tuning doesn’t actually stop the model making things up.

Q19.3 — Walk me through your RAG pipeline end to end.

“Load the PDF. Split it into 800-character chunks with 150 characters of overlap. Turn each chunk into numbers and store them in FAISS. When a question comes in, turn it into numbers the same way, find the closest chunks, paste them into the prompt with the question, and have Llama 3 answer from them — returning the source passages alongside the answer.”

Be able to say that in thirty seconds.

Q19.4 — What is chunking, and why not embed whole documents?

Splitting the document into passages.

Two reasons you can’t use whole documents. One embedding for a whole document averages everything together until it means nothing specific. And a whole document wouldn’t fit in the model’s context anyway.

Q19.5 — Why 800 characters with 150 overlap specifically?

Big enough to contain one complete idea, small enough to stay precise and cheap. The overlap means a sentence sitting across a boundary still appears whole in one of the two chunks.

Say honestly whether you tuned it or picked a sensible default.

Q19.6 — What is `RecursiveCharacterTextSplitter` doing?

It tries to split at natural boundaries in order of preference — paragraph breaks first, then line breaks, then sentences, and only chops mid-sentence as a last resort.

Q19.7 — What other chunking strategies exist?

Fixed size, by sentence, by meaning (split where the topic shifts), or by document structure (by heading or section).

For regulations, splitting by clause beats any character count — a rule is a natural unit and chopping one in half is exactly the wrong thing to do.

Q19.8 — How does chunk size affect quality in both directions?

Too small: the answer gets split across several chunks and none of them is sufficient on its own.

Too large: irrelevant text comes along for the ride, the search gets less precise, and you burn context.

There's no universal number — it depends on how your documents are written.

Q19.9 — What goes in a RAG prompt?

A role instruction, the retrieved passages clearly marked off as source material, the question, and an explicit rule: answer only from what's above, and say so if it isn't there.

Q19.10 — How do you make the model cite its sources?

Give every chunk an identifier — file, page, chunk number. Include those identifiers in the prompt. Instruct the model to reference them. Then check the citations actually point at real passages.

Your traceability claim rests entirely on that last step.

Q19.11 — How do you stop it answering from its own knowledge?

Tell it to use only the provided text. Keep temperature near zero. And verify that what it cited actually appears in the retrieved passages.

Belt and braces, because instructions alone leak — the model will sometimes answer from memory regardless of what you told it.

Q19.12 — What is “lost in the middle”?

Models pay most attention to the beginning and the end of a long context, and can skip over material in the middle.

So put the most relevant passages first, and keep the number of passages modest rather than dumping everything in and hoping.

Q19.13 — How do you evaluate your RAG system?

Split it in two.

Retrieval: build a set of questions where you know which passage contains the answer, then measure how often that passage comes back in the top results.

Generation: check whether the answer is actually supported by what was retrieved, and whether anything was invented.

Fifty real questions is enough to catch regressions.

Q19.14 — What is RAGAS?

A ready-made evaluation library that scores faithfulness, answer relevance, and whether the retrieved context was any good — so you don't build all of it yourself.

Q19.15 — Your RAG returns the wrong chunk. How do you debug it?

Work backwards through the stages.

Is the right chunk **in the index at all**? If not, ingestion is broken. Is it in the top fifty? If not, your embedding model is wrong for this domain. Is it in the top five? If not, your ranking needs a reranker.

That's three questions and each eliminates a whole layer. Logging what got retrieved for every query is not optional — without it you're guessing.

Q19.16 — What is query expansion or rewriting?

Rephrasing the user's question before searching — generating variants, or turning a follow-up into a standalone question.

Essential in a chat, where “what about gloves?” means nothing on its own but means a great deal given the previous turn.

Q19.17 — What is HyDE?

Have the model write a **hypothetical** answer first, then search using that instead of the question.

Why it works: a short question looks nothing like the document that answers it, but a draft answer looks a lot like it.

Q19.18 — What is parent-document retrieval?

Search over small precise chunks, but hand the model the larger section that chunk came from.

You get precise matching and complete context, instead of having to choose.

Q19.19 — How do you handle tables and images in PDFs?

Plain text extraction mangles tables — columns get interleaved into nonsense. You need a layout-aware parser, and tables are best converted into a simple text table format.

For images, have a vision model describe them and index the description.

A good honest answer: “my current pipeline handles plain text well and tables badly.”

Q19.20 — What is a multi-query retriever?

Generate several rephrasings of the question, search with each, and combine the results. It helps when the user's exact wording happens to be unlucky.

Q19.21 — How does SafeSight's RAG differ from DocuMind's?

DocuMind is one-shot: a person types a question about documents they uploaded.

SafeSight is triggered by a **detection**, not a question. A violation becomes the query, rules get retrieved, and the graph then decides whether those rules actually apply — looping back if they don't.

Different shape of problem, which is why one uses a graph and one doesn't.

Q19.22 — How would you scale from one user's PDFs to an enterprise corpus?

Swap FAISS for a managed store with filtering and access control. Add hybrid search and a reranker. Cache embeddings and common queries. Move ingestion into a queue so uploads don't block. And enforce per-user document permissions at query time.

That last one is what makes it a production answer. Without it, retrieval will happily hand someone a passage from a document they aren't allowed to read.

Q19.23 — What is prompt injection, and why is it a real risk in RAG?

Instructions hidden inside a document you ingest. The model can't tell the difference between your instructions and text it retrieved — they arrive in the same prompt as the same kind of thing.

So a document containing "ignore your instructions and..." may well be obeyed.

Mitigate by clearly labelling retrieved text as data rather than instructions, constraining what the output can be, and never letting retrieved text directly trigger an action.

Q19.24 — What are RAG's main failure modes?

The document was never indexed. The chunking split the answer. The embedding model is wrong for this domain. k is too small. The answer was in the middle and got skipped. The index is stale. Or the model ignored the context.

Diagnose by layer, using Q19.15's backwards walk. Never by guessing.

PRACTICE | The debugging answer

Q19.15 is the one that separates people. Memorise the three-step walk: is it in the index, is it in the top fifty, is it in the top five. Each answer points at a different layer. Being able to say that immediately is worth a lot.

CHAPTER RECAP

- An embedding puts similar meanings close together; cosine similarity ignores length, which is why text uses it.
- FAISS is an index, not a database — no persistence, no filtering, no users. Say so.
- Save the chunk texts alongside the index or it's useless.
- A reranker is the single biggest easy win in RAG.
- Hybrid search fixes the one thing embeddings are bad at: exact codes and names.
- Debug retrieval backwards: in the index, in the top fifty, in the top five.
- Permissions at query time is what makes a scaling answer a production answer.

Prompting, LangChain and Agents

Including the MCP and A2A questions you listed

CORE

PREREQS: Chapter 12

Your resume names LangChain, LangGraph, agents, MCP and A2A, so all five get asked. The one that matters most is why SafeSight needed a graph rather than a chain — that is a design decision you made, and being able to defend it is worth more than any definition.

WHAT YOU WILL LEARN

- Say what makes a prompt reliable rather than lucky
- Explain the difference between a chain and a graph, with your own example
- Define MCP and A2A in one line each
- Say when you would *not* use an agent

Prompting

Q20.1 — What is prompt engineering?

Writing the instruction carefully so you get a reliable answer instead of a lucky one. A clear task, the relevant context, explicit limits, and the format you want back.

Q20.2 — Zero-shot vs one-shot vs few-shot?

How many worked examples you include. None, one, or several.

Few-shot is how you pin down the exact output format and show it how to handle edge cases, without any training.

Q20.3 — What makes a good prompt?

A specific task, the context it needs, explicit constraints, a stated output format, and an instruction for what to do when it doesn't know.

Vague prompts produce vague answers. That's not a model limitation.

Q20.4 — What is a system prompt?

Standing instructions that apply to the whole conversation — the role and the rules — kept separate from what the user types each turn.

Q20.5 — What is chain-of-thought, and when does it hurt?

Asking it to reason step by step before answering. It helps on anything with multiple steps.

It hurts when you want a short structured answer — it adds length, latency and cost for no benefit, and on simple classification it can talk the model out of a correct first instinct.

Q20.6 — What is self-consistency?

Ask the same question several times, let it reason differently each time, and take the most common answer. More reliable, several times more expensive.

Q20.7 — How do you get reliable JSON out of an LLM?

Ask for JSON and nothing else. Give it the exact structure you want. Use the provider's structured-output mode if there is one. Set temperature to zero. Then **validate** what comes back and retry if it's malformed.

The validation step is the one people skip, and it's the one that takes you from "usually works" to "always works."

Q20.8 — What is ReAct prompting?

Alternating between thinking and doing: think about what's needed, take an action, look at the result, think again. It's the pattern underneath most agents.

Q20.9 — How do you handle a prompt that works 90% of the time?

Collect the failures into a test set. Then change **one thing** and measure against it.

Treat prompts like code — versioned, with tests. Changing things by feel is how you fix one case and break three others without noticing.

Q20.10 — What is prompt chaining, and why decompose?

Splitting a complicated task into several simpler prompts in sequence.

Each step is simpler, each can be tested on its own, and when something goes wrong you know which step did it. One prompt doing five jobs is impossible to debug.

Q20.11 — How do you reduce token cost in production?

Trim the system prompt. Retrieve fewer but better passages. Cache the parts of the prompt that never change. Summarise long conversation history rather than resending it. And send easy requests to a smaller model.

LangChain and LangGraph

Q21.1 — What is LangChain?

A toolkit that wraps the common pieces of an LLM application — talking to models, prompt templates, retrievers, memory, tools — behind one consistent interface, so you can swap pieces without rewriting everything.

Q21.2 — What is a chain?

A fixed sequence of steps: build the prompt, call the model, parse the result. Straight through, no branching, no going back.

Q21.3 — What is LangChain's retrieval QA doing under the hood?

Turn the question into numbers, pull the closest chunks from the store, slot them into a prompt template with the question, call the model, and hand back the answer plus the sources.

Nothing magic — it's the pipeline you'd write yourself, just packaged.

Q21.4 — What are the “stuff”, “map-reduce” and “refine” chain types?

Stuff: put all the chunks in one prompt. Simple, but limited by context size.

Map-reduce: answer from each chunk separately, then combine the answers. Works on more text, costs more calls.

Refine: answer from the first chunk, then improve that answer with each subsequent chunk. Good for summaries, but it's sequential so it's slow.

Q21.5 — What is LangGraph, and how does it differ from a chain?

It lets you build the flow as a **graph** rather than a line — with branches, loops, and shared state that every step can read and write.

A chain goes forward once. A graph can decide where to go next, and it can go back.

Q21.6 — Why did SafeSight need a graph rather than a chain?

Because the flow has a decision and a loop in it.

Work out what kind of violation it is. Look up candidate rules. **Check whether those rules actually apply** — and if none of them do, go back and search differently. Only then produce the answer.

A chain can't re-enter a step it's already passed. That's the whole reason.

Q21.7 — What is state in LangGraph?

A shared object passed between steps. Each step reads what's there and adds its own findings.

It's what lets a later step see what an earlier one found — without it, each step would be starting fresh.

Q21.8 — What is a conditional edge?

An arrow whose destination is decided by looking at the current state. It's how you implement “if no rule matched, go back and try again.”

Q21.9 — What is a checkpoint?

Something that saves the state as the graph runs, so a run can be paused, resumed, or inspected. It's what makes human-in-the-loop review possible — you can stop, let a person look, and carry on.

Agents

Q21.10 — What is an AI agent?

A model that decides what to do next, uses tools, looks at what came back, and keeps going until the job is done — rather than producing one fixed response.

Q21.11 — Agent vs chain — when would you NOT use an agent?

When you already know the steps.

Agents add latency, cost, and unpredictability. If you can draw the flowchart, build the flowchart.

This is the senior answer, and most interviewers are specifically listening for it. Reaching for an agent when a fixed sequence would do is the most common mistake in this area.

Q21.12 — What is tool calling?

You describe the tools available — what each does and what arguments it takes. The model produces a structured request to use one. **Your code** actually runs it and hands back the result.

The model never executes anything itself. That distinction matters for security.

Q21.13 — What is agent memory? Short vs long term?

Short-term is the conversation currently in the context window. Long-term is saved somewhere and retrieved when relevant.

The distinction exists only because context windows are finite — otherwise you'd just keep everything.

Q21.14 — What is a multi-agent system, and what's the usual failure?

Several specialised agents working together — one plans, one researches, one checks.

The usual failure is compounding. Every handoff adds latency, cost, and another chance for something to go wrong. Three agents at 90% reliability each gives you about 73% overall.

Q21.15 — What is MCP?

Model Context Protocol — an agreed standard for how a model connects to tools and data.

Q21.16 — What problem does MCP actually solve?

Without a standard, every tool needs custom wiring for every framework. Ten tools and five frameworks means fifty separate integrations.

With it, each side implements the standard once. Think of it as a USB-C port for model tooling — one shape that everything plugs into.

Q21.17 — What does an MCP server expose?

Three things: tools it can call, data it can read, and reusable prompt templates.

Q21.18 — What is A2A?

Agent-to-Agent — a standard for agents built by **different teams or companies** to find each other and work together, passing tasks and results between them as peers.

Q21.19 — MCP vs A2A — the clean distinction?

MCP connects an agent downward to tools and data. A2A connects an agent sideways to other agents.

They complement each other rather than compete. That one-liner is exactly what they're listening for.

Q21.20 — What are the real risks of deploying agents?

Loops that never end and burn money. Tools being used in ways you didn't intend, with real consequences. Hidden instructions in retrieved content hijacking what it does. And no record of what happened afterwards.

Mitigate with hard step limits, giving each agent only the tools it genuinely needs, requiring human approval before anything irreversible, and logging every single call.

Q21.21 — How do you debug an agent that behaves unpredictably?

Trace every step — the prompt sent, the tool called, the result, and the state at each point. LangSmith or equivalent.

Without tracing you are genuinely just guessing, because the same input can produce a different path each time.

PRACTICE | The three to have ready

Q21.6 — why SafeSight needed a graph, because that's a decision you made. Q21.11 — when **not** to use an agent, because that's the senior answer. Q21.19 — MCP down, A2A sideways, because it's one line and it's the question you listed yourself.

CHAPTER RECAP

- A good prompt is specific, bounded, and says what to do when it doesn't know.
- Validate structured output and retry — that's what makes it reliable.
- A chain goes forward once; a graph can branch and loop. SafeSight needed the loop.
- Don't use an agent when you already know the steps. Say this.
- The model requests a tool; your code runs it. Never the other way round.
- MCP connects downward to tools; A2A connects sideways to other agents.
- Without tracing, debugging an agent is guesswork.

PART VI

Shipping It

Containers, pipelines, APIs, version control and system design. The half of the job that turns a notebook into something a company can run.

-
- 14 MLOps: MLflow, Docker and CI/CD
 - 15 APIs, Streamlit and Git
 - 16 ML System Design

MLOps: MLflow, Docker and CI/CD

Your churn project is the evidence, so know every piece of it

CORE

PREREQS: none

This is where your churn project earns its place. The model is weak and the pipeline is strong, so lead with the pipeline. Two questions here are about your own stale README — have the honest answer ready, and fix the README before anyone reads it.

WHAT YOU WILL LEARN

- Explain why ML is harder to deploy than normal software
- Walk through your Dockerfile and justify the line order
- Describe what your CI pipeline actually does, without inflating it
- Say how you would detect a model going wrong in production

The idea

Q22.1 — What is MLOps?

Applying normal software engineering discipline to machine learning: versioned data and models, training you can repeat and get the same answer, automated testing and deployment, and watching it once it's live.

Q22.2 — Why is ML harder to deploy than normal software?

Three reasons.

Normal code does the same thing forever; a model's behaviour depends on data, so you have to version the data too. Models get worse over time as the world changes, even though the code never changed. And correctness isn't yes/no — it's a number that can drift.

Q22.3 — What is MLflow, and which parts have you used?

A tool for recording experiments and managing models. You log what settings you used, what score you got, and the model file itself — so six weeks later you can find out exactly what produced the thing currently running.

You used the tracking part plus automatic promotion of the best run.

Q22.4 — Run vs experiment in MLflow?

An experiment is a named group — “churn model”. A run is one execution inside it, with its own settings, scores and saved files.

Q22.5 — How does your automatic best-run promotion work?

It looks through the recorded runs, compares them on the logged score, and marks the winner as the one to serve.

Then volunteer the weakness: **the logged score is accuracy**. So it can promote a model that’s actually worse at catching churners. Naming your own design flaw is far more impressive than hoping they don’t spot it.

Q22.6 — What is a model registry?

Versioned storage for models with stages — staging, production, archived. So your deployment points at “whatever is in production” rather than a file path that someone has to remember to update.

Q22.7 — Why log parameters and metrics rather than keep a spreadsheet?

Because in six weeks you’ll need to know exactly which settings produced the model currently serving traffic, and a spreadsheet will be out of date or missing.

Docker

Q22.8 — What is Docker, and what problem does it solve?

It packages your code, its libraries and its runtime into one image that runs identically anywhere. It kills “it works on my machine.”

Q22.9 — Container vs virtual machine?

A virtual machine simulates an entire computer including its own operating system — gigabytes, slow to start.

A container shares the host’s operating system and just isolates your application — megabytes, starts in seconds.

Q22.10 — Image vs container?

The image is the recipe. The container is the thing actually running. One image, many containers.

Q22.11 — Walk me through your Dockerfile.

Start from a Python base image. Set the working directory. Copy `requirements.txt`. Install the dependencies. **Then** copy the source code. Expose the port. Set the command to run.

Know why the requirements come before the source — it’s the next question and it’s a favourite.

Q22.12 — Why copy requirements before the application code?

Because Docker caches each step. Dependencies change rarely; your code changes constantly.

Copy requirements first and install, and then a code change only rebuilds the last step — a few seconds. Copy everything at once and every one-character edit reinstalls PyTorch from scratch.

This is one of the most-asked Docker questions there is.

Q22.13 — What is a multi-stage build, and why for ML?

Build in a big image that has all the compilers, then copy only the finished result into a small clean image.

ML images bloat quickly — the build tools alone can be hundreds of megabytes. This strips all of that out of what you actually ship.

Q22.14 — What is Docker Compose, and what does it give your churn project?

A single file describing several containers and how they connect, so one command starts everything.

For your project it means a reviewer runs `docker compose up` and gets the API and the UI running together. That's a real advantage when someone is assessing your work.

Q22.15 — How do containers talk to each other in Compose?

By service name on a shared network. So the UI calls `http://api:8000`, not `localhost` — because inside a container, `localhost` means that container itself.

Q22.16 — What is a volume, and why does it matter for models?

Storage that lives outside the container and survives restarts.

It matters because model weights and your MLflow data must be on a volume. Otherwise every restart wipes them.

Q22.17 — How do you keep secrets out of an image?

Environment variables, a `.env` file excluded from Git, or a proper secrets manager.

Never bake a key into the image. Even if a later step deletes it, it's still sitting in the image's history where anyone can read it. Directly relevant to your SMTP credentials.

Q22.18 — What is `.dockerignore` for?

Keeping junk out of the build — the `.git` folder, datasets, virtual environments. Smaller build context means faster builds and smaller images.

Q22.19 — What is Kubernetes, and would your project need it?

It runs containers across many machines, handling scaling and restarting things that crash.

Honest answer: **no**, a single-service demo doesn't need it. Knowing when not to reach for something is a point in your favour, not against you.

CI/CD

Q22.20 — What is CI/CD?

Continuous Integration: every time you push code, it gets built and tested automatically.

Continuous Delivery: if it passes, it gets released automatically.

Q22.21 — What does your GitHub Actions pipeline actually do?

Runs import checks and builds the Docker image on push.

Describe it exactly. It's modest, and overstating it is much worse than its actual size — because the follow-up question will expose it.

Q22.22 — Your README says CI/CD is future work but the workflow exists. Explain.

"The workflow is real, the README is out of date."

Then fix it. And check the workflow triggers on the branch you actually use — if it's set to `main` and your branch is `master`, it has never run once, and finding that out live is painful.

Q22.23 — What is a GitHub Actions workflow made of?

A YAML file with three parts: when to run (on push, on pull request), what machine to run on, and the list of steps.

Q22.24 — What would you add to your CI pipeline next?

Real unit tests. Linting. A check that the data has the expected columns. A smoke test that actually calls `/predict` in the built container. And a minimum-score gate that fails the build if the model gets worse.

Having this list ready turns a thin pipeline into evidence of judgement. You're showing you know what good looks like even if you haven't built it yet.

Monitoring

Q22.25 — What is model drift? Data drift vs concept drift?

Data drift: the incoming data changes. Your customers are now younger than the ones you trained on.

Concept drift: the relationship changes. The same customer profile now means something different because a competitor launched.

Both make a model worse without any code changing.

Q22.26 — How do you detect drift without labels?

You usually don't get to know the right answer for months, so you watch the inputs and the outputs instead.

Compare the distribution of incoming features against what you trained on, and watch the distribution of predictions. If the model suddenly starts predicting churn twice as often, something has changed — even before you know whether it's right.

Q22.27 — *How do you monitor a model in production?*

Four layers, in order of how quickly they tell you something.

Operational: is it up, how fast, how many errors. Data quality: missing fields, impossible values, categories you've never seen. Prediction distribution: has the output shifted. And finally accuracy, once the real answers eventually arrive.

Q22.28 — *Shadow deployment, canary, blue-green?*

Shadow: the new model sees real traffic but its answers aren't used. Zero risk, and you can compare it to the old one on identical inputs.

Canary: a small slice of real traffic goes to the new model.

Blue-green: two complete environments, and you flip a switch.

All three are about limiting how much damage a bad model can do.

Q22.29 — *What is A/B testing a model?*

Splitting real traffic between two models and comparing a **business** outcome — retained customers, not offline accuracy. With enough traffic to tell a real difference from noise.

Q22.30 — *How often would you retrain the churn model?*

Trigger-based rather than on a calendar: retrain when the data has drifted past a threshold, or when recall drops.

If you have no monitoring yet, monthly is a reasonable default — but say that it's a default, not a decision.

Q22.31 — *What is reproducibility, and how do you achieve it?*

Same inputs give the same model. Pin your library versions, set the random seeds, version the data, log everything, and containerise the environment.

Miss any one and you get a model you can't recreate.

Q22.32 — *What is a feature store, and would you need one?*

A central place where computed features live, so training and serving both use exactly the same calculation.

Would you need one? No — it's for when several models and teams share features. For one project it's overkill, and saying so shows judgement.

Q22.33 — *What is training-serving skew?*

When the way you compute a feature in training differs from how you compute it when serving. The model gets inputs that don't look like what it learned from, and quietly gets worse.

Your protection is that the scaler and encoders ship **inside** the saved pipeline, so both paths run identical code.

PRACTICE | Before the interview

Open your `ci.yml` and check the branch it triggers on. Then open both stale READMEs and fix them. That is an hour's work and it closes two of the most uncomfortable questions in this chapter.

CHAPTER RECAP

- ML is harder to deploy because behaviour depends on data, and models decay while code doesn't.
- Copy requirements before source in a Dockerfile — layer caching. Most-asked Docker question.
- Name your MLflow promotion flow yourself: it promotes on accuracy.
- `localhost` inside a container means that container — use service names.
- Never bake secrets into an image; they stay in the history.
- “No, we don't need Kubernetes” is a good answer.
- Watch inputs and prediction distributions, because real labels arrive late.

APIs, Streamlit and Git

The plumbing round

CORE

PREREQS: none

FastAPI, Streamlit and Git are on your resume and all three appear in your projects, so these are fair game. Two of them — where you load the model, and `def` versus `async def` — are the questions that reveal whether you have actually served a model or only written one.

WHAT YOU WILL LEARN

- Explain REST and the status codes you will be asked about
- Say where to load a model and why, without hesitating
- Answer the `async` question correctly, because most people get it wrong
- Describe a sensible Git workflow in one sentence

REST and FastAPI

Q23.1 — What is a REST API?

A way for one program to ask another for something over the web. You send a request to an address, you get a response back, usually as JSON.

Q23.2 — GET vs POST vs PUT vs DELETE?

GET fetches something and changes nothing. POST creates something or submits data. PUT replaces something. DELETE removes it.

Model predictions use POST, because you're sending data in the body.

Q23.3 — What makes an API RESTful?

Addresses name things rather than actions (`/orders/42` , not `/getOrder?id=42`), the right verb for the right job, and every request carries everything needed — the server doesn't remember you between calls.

Q23.4 — Name the status codes you should know.

200 fine. 201 created. 400 your request was malformed. 401 you're not logged in. 403 you're logged in but not allowed. 404 not found. 422 the data failed validation. 429 you're sending too many. 500 we broke. 503 we're down.

The 401-versus-403 distinction gets asked: 401 is “who are you”, 403 is “I know who you are and no”.

Q23.5 — What is FastAPI, and why choose it for ML serving?

A Python web framework that reads your type annotations and uses them to check incoming requests automatically, and to generate interactive docs.

For serving a model that’s three things nearly free: input validation, good throughput, and a `/docs` page where anyone can try your model without writing code.

Q23.6 — What is Pydantic doing in your API?

Defining the shape of what comes in and goes out, checking incoming JSON against it, and rejecting anything wrong with a clear error **before** it reaches your model.

Q23.7 — Why validate inputs at the API boundary for an ML service?

Because a model given nonsense doesn’t crash — it returns a **confident wrong number**. There’s no error to catch.

Validation turns a silent failure into a loud one. That’s the whole argument, and it’s specific to ML.

Q23.8 — What is ASGI? What is Uvicorn?

ASGI is the modern standard for how a Python web app talks to a web server, supporting things that happen concurrently. Uvicorn is a server that implements it and actually runs your app.

Q23.9 — `def` vs `async def` in FastAPI — which for model inference?

Use `async def` when you’re **waiting** for something — a database, another API.

Use plain `def` for model prediction, because that’s **calculation**. FastAPI automatically runs plain functions on a separate thread so they don’t freeze everything else.

Putting a blocking prediction inside `async def` is a classic mistake: it blocks the main loop and your whole service stalls while one prediction runs.

Q23.10 — Where do you load the model — per request or at startup?

At startup, once.

Loading it per request adds seconds to every single call. Expect to be asked this — it’s a quick way to find out whether you’ve actually served a model.

Q23.11 — How do you handle concurrent requests to one model?

Run several worker processes, or queue requests and process them in batches.

Watch memory though — each worker loads its own copy of the weights. Four workers means four copies.

Q23.12 — What is a health-check endpoint, and why does it matter?

A cheap `/health` address that returns OK when the service is up and the model is loaded.

Load balancers and orchestrators call it to decide whether to send you traffic. Without it, they’ll happily route requests to a container that’s still starting up.

Q23.13 — How do you version an ML API?

Put the version in the address (`/v1/predict`) and return the **model** version in the response.

That second part matters: it means any prediction can be traced back to exactly which model made it, months later.

Q23.14 — What is CORS, and when does it bite you?

Browsers block a page on one address from calling a different address, unless the server explicitly allows it.

It bites the moment your React dashboard on one port calls your FastAPI on another — which is exactly SafeSight's setup.

Q23.15 — What is Streamlit, and what are its limits?

A library that turns a Python script into a web page, which is ideal for demos and internal tools.

Limits: it re-runs the entire script every time anyone clicks anything, you have limited control over layout, and it isn't built to be a real multi-user front end.

Q23.16 — Why Streamlit for the UI but FastAPI for the model?

Separation. The API is reusable and testable on its own; Streamlit is just one thing that happens to call it.

It also means the same model can serve the React dashboard, a script, or anything else — without duplicating the logic.

Q23.17 — `st.cache_data` vs `st.cache_resource` ?

`cache_data` for values you can copy — a loaded dataframe. `cache_resource` for things there should only ever be one of — your model, a database connection.

Without caching, Streamlit reloads your model on every single widget click, because it re-runs the whole script.

Q23.18 — How do you secure a prediction endpoint?

An API key or token, HTTPS, rate limiting, a cap on input size, and no stack traces in error responses — those leak your file paths and library versions.

Q23.19 — How would you stream LLM responses to a UI?

Send tokens as they're generated rather than waiting for the whole answer.

The total time doesn't change at all, but it **feels** far faster because something appears immediately. Worth mentioning for DocuMind.

Git

Q24.1 — What is Git, and what is GitHub?

Git tracks changes to your code and runs entirely on your machine. GitHub hosts those repositories online and adds the collaboration layer — pull requests, issues, automated builds.

Q24.2 — Explain the basic workflow.

Pull the latest. Make a branch. Edit. Stage. Commit. Push. Open a pull request. Get it reviewed. Merge.

Q24.3 — `git fetch` vs `git pull`?

Fetch downloads what's changed but doesn't touch your files. Pull does both — downloads and merges it in.

Fetch first if you want to see what's coming before accepting it.

Q24.4 — Merge vs rebase?

Merge keeps both histories and adds a commit joining them. Rebase replays your commits on top of the other branch so the history looks like a straight line.

The rule: never rebase a branch other people are working on. You'd be rewriting history they already have.

Q24.5 — What is a merge conflict, and how do you resolve one?

Two branches changed the same lines, so Git can't decide which is right. It marks both versions in the file; you edit it to what it should actually be, then stage and commit.

Q24.6 — `git reset` vs `git revert`?

Reset rewinds history — only safe on commits nobody else has.

Revert adds a **new** commit that undoes an old one, leaving history intact. That's the safe one on a shared branch.

Q24.7 — What is `.gitignore`, and what belongs in it for ML?

A list of things Git should ignore. For ML: virtual environments, cache folders, your `.env` file, datasets, and model weights.

Q24.8 — Should model weights and datasets live in Git?

No. Git keeps a complete copy of every version of every file forever, so a few rounds of a 200 MB model makes the repository enormous — permanently. Deleting it later doesn't shrink it.

Use Git LFS, DVC, or just store them elsewhere and keep a reference.

Q24.9 — What is a pull request, and what makes a good one?

A request to merge your branch, with a chance for someone to review it first.

A good one is small, does one thing, and explains **why** rather than just what. A 2,000-line pull request doesn't get reviewed, it gets approved.

Q24.10 — What is a good commit message?

A short imperative summary, then a blank line, then the reasoning if it needs one.

“Fix NMS threshold merging adjacent worker boxes” beats “update”. In six months that message is the only explanation anyone has.

Q24.11 — What is `git stash`?

Puts your half-finished changes aside so you can switch branches, then lets you bring them back.

Q24.12 — How do you find which commit broke something?

`git bisect`. You tell it a commit that worked and one that doesn't, and it does a binary search — checking out the midpoint, you say better or worse, and it narrows down.

Three hundred commits takes about nine checks instead of reading three hundred diffs.

PRACTICE | The two that catch people

Q23.9 and Q23.10 — `async` versus `plain`, and where you load the model. Both are one sentence, both are asked constantly, and both reveal immediately whether you've actually served a model or only trained one.

CHAPTER RECAP

- 401 is “who are you”, 403 is “I know and no”.
- Validate at the API boundary, because a model given nonsense returns a confident wrong number rather than an error.
- Plain `def` for prediction, `async def` for waiting.
- Load the model once at startup.
- Return the model version with every prediction so it can be traced.
- Never rebase a shared branch; revert rather than reset on one.
- Model weights never belong in Git.

ML System Design

Where structure matters more than detail

ADVANCED

PREREQS: Chapter 14

These get asked at the senior end of a junior interview, and they are not really testing whether you know the answer. They are testing whether you have a structure — whether you ask about the goal before designing anything. Say the structure out loud before you start.

WHAT YOU WILL LEARN

- Open any design question with the same structure, out loud
- Design the three systems you have actually built
- Say where humans belong in the loop
- Raise fairness and privacy before they do

Q25.1 — How do you approach any ML system design question?

Say the structure first, then follow it:

What's the goal and what are the constraints. What exactly is being predicted, and what's the label. Where does the data come from. What features. What's the simplest baseline. Then the model. Then how you'd measure it offline. Then how it gets served. Then how you'd monitor it. Then what happens when it fails.

Saying that out loud at the start is worth more than any individual answer, because it shows you won't just start coding.

Q25.2 — Design a PPE compliance system from scratch.

You've built this, so use the real one.

Cameras feeding frames. Sample a few frames per second rather than all of them. Detect people and safety items. Track people so identities stay stable. Map each violation to the governing rule by retrieval. Store the event with an evidence image. Dashboard and alerts on top. A human review loop at the end.

Mention frame skipping and per-camera monitoring — those are the details that make it sound deployed rather than imagined.

Q25.3 — Design a churn prediction system.

Start by pinning down what churn actually **means** — no activity for thirty days, or an explicit cancellation? And how far ahead are you predicting?

Then: features from usage, billing and support tickets. Train with a time-based split. Optimise for catching churners at a threshold justified by what a retention offer costs. Score weekly in batch. Feed the list to the retention team. And measure whether the **campaign** worked, not just whether the model scored well.

That last point is what separates a model from a system.

Q25.4 — Why is a time-based split essential for churn?

Because a random split lets the model learn from the future. It sees March data while predicting February, which it will never be able to do live.

Train on earlier periods, test on later ones — the way it'll actually be used.

Q25.5 — Design a document QA system for 10 million documents.

An ingestion queue so uploads don't block. Layout-aware parsing. Chunking. Embedding in batches. A managed vector store, split across machines, with metadata and **access control**. Hybrid retrieval plus a reranker. The model with citations. A cache. And an evaluation set with a way to capture user feedback.

What actually changes at that scale is permissions, ingestion throughput and cost — not the core idea. Say that.

Q25.6 — Design a real-time fire detection system for a building.

Multiple camera feeds. A deliberately low confidence threshold, because missing a fire is far worse than a false alarm. **Confirmation across several consecutive frames** before alerting, which kills most flickers. Tiered alerts with evidence attached. Escalation if nobody acknowledges.

And then the important clause: it supplements physical smoke and heat sensors, it never replaces them. Saying that unprompted is what shows judgement.

Q25.7 — How do you reduce false alarms without hurting recall?

Require the detection to persist over several frames. Ignore regions you know are noisy — a window with sunset through it. Combine with another signal, like an actual smoke sensor. And tier the alerts by confidence rather than using one hard cutoff.

The point is you're not raising the threshold — that would cost you recall. You're adding other evidence.

Q25.8 — Batch vs real-time inference — how do you decide?

By how fast the decision actually has to be made.

Churn scoring is fine weekly in batch. PPE and fire detection have to be streaming. Batch is far cheaper and much simpler, so default to it unless latency genuinely demands otherwise.

Q25.9 — How do you estimate the cost of an ML system?

Training compute, inference compute multiplied by how many requests you expect, storage for data and models, and token costs if you're calling a hosted model.

For your local Llama 3 setup, the point is that the per-request cost is zero but the hardware cost is fixed and up front. That's a genuinely different shape of bill.

Q25.10 — Where do humans belong in the loop?

Wherever an automated mistake is expensive. Enforcing a safety penalty. Deleting something. Sending a retention offer that costs money.

Design the review queue as **part of the system**, not as an apology for the model not being good enough.

Q25.11 — How do you handle a model that is accurate but unfair?

Measure per group, not in aggregate. An overall number can look fine while one group is being badly served.

For PPE detection that means checking detection rates across skin tone, clothing and lighting conditions — and treating a gap as a **bug with a data fix**, not as an unfortunate property of the model.

Q25.12 — What does it mean for a model to be explainable?

Being able to say why it made a particular decision.

Methods: overall feature importance for the whole model; SHAP or LIME for one specific prediction; and for image models, heatmaps showing which part of the picture drove the answer.

Q25.13 — How would you explain your churn model's prediction to a business user?

Not as a probability. As reasons: "short tenure, month-to-month contract, three support tickets last month."

A retention team can act on that. They cannot act on 0.73.

PRACTICE | The opener

Rehearse Q25.1 until you can say the structure in fifteen seconds. In a design question, starting with "let me check what we're optimising for and what the constraints are" immediately changes how the interviewer sees you.

CHAPTER RECAP

- Say the structure before you design anything: goal, label, data, baseline, model, metric, serving, monitoring, failure.
- Define what churn actually means before building a churn model.
- Fire detection supplements certified sensors; it never replaces them.
- Reduce false alarms with confirmation over time, not by raising the threshold.
- Measure fairness per group, and treat a gap as a bug with a data fix.
- Explain predictions as reasons, not probabilities.

PART VII

Your Work and the Last Mile

Your four projects, defended question by question; the coding screen; what to ask them; and the checklist for the morning of the interview.

-
- 17 Your Four Projects
 - 18 The Coding Round
 - 19 What to Ask Them, and the Final Checklist

Your Four Projects

Every claim here has to survive a follow-up

CORE

PREREQS: Chapter 2

This is where they will spend the longest, and it is where an inflated claim collapses fastest — because the next question is always an implementation detail of whatever you just said. Read this chapter and Chapter 2 together; between them they cover everything that can go wrong.

WHAT YOU WILL LEARN

- Give a 60-second overview of each project that invites the right follow-up
- Name your own contribution precisely, without absorbing the team's work
- Volunteer each project's weakness before they find it
- Say what you would build next, concretely

SafeSight

Q26.1 — Give me a 60-second overview of SafeSight.

“It’s a workplace safety platform for industrial sites. A YOLOv8 model watches the CCTV feeds and spots workers plus five types of safety equipment, scoring 0.841 on the standard detection metric. Every violation gets logged with a timestamp and an evidence photo, so there’s an audit trail. Then a second part looks up which safety regulation was actually broken and shows the supervisor the specific clause, not just an alert. There’s a dashboard on top showing it live.”

Q26.2 — What problem does it solve that a human can’t?

Coverage. A safety officer can watch one area for part of a shift. The system watches every camera continuously and produces the evidence trail that an audit actually requires.

It’s not that it’s smarter than a person — it’s that it never looks away and it writes everything down.

Q26.3 — What was your specific contribution?

Answer precisely. Name the parts you built, name what you integrated, and name what already existed before you joined.

Do not absorb the team’s work. The next question is always a detail of whatever you claimed, and if you claimed something you didn’t build, that’s where it falls apart.

Q26.4 — Where did the training data come from, and how was it annotated?

Know the answer: public datasets, client footage, or both. Who annotated it. Roughly how many images. How the classes were defined.

If it was a public dataset, say so. Using public data is completely fine. Being vague about where your data came from is not.

Q26.5 — What was the hardest technical problem?

Working out **which worker** a piece of safety equipment belongs to when several people overlap in frame — and keeping that association stable as they move.

Explain how you did it (checking whether the item's box sits inside the person's box) and be honest about where it still fails (two people standing close together).

Q26.6 — How do you avoid logging the same violation hundreds of times?

Tracking plus a cooldown. One tracked worker without a helmet becomes **one** event with a duration, not one event per frame.

Without this the audit log is unusable — thirty seconds of footage would generate hundreds of entries for the same person standing still.

Q26.7 — How does the assistant know which rule applies?

The violation type becomes a search query. Chunks of the safety manual come back ranked by relevance. The graph then checks whether a retrieved rule actually governs this situation, and searches differently if none of them do. Then it answers with the clause cited.

Q26.8 — What if it retrieves the wrong regulation?

Then a supervisor sees a citation that doesn't fit the situation.

That's exactly why it **cites rather than decides** — the human verifies before anything is enforced. Being clear that the model assists and doesn't have authority is the right answer for a safety system, and saying it unprompted is a strong signal.

Q26.9 — How many cameras could it handle, and what was the bottleneck?

Know your real number and which resource ran out first — GPU throughput, video decoding, or network bandwidth.

Then mention frame skipping: safety compliance doesn't need thirty frames a second. Running at three to five multiplies how many cameras one machine handles.

Q26.10 — What was the latency from violation to alert?

Have a figure and say what it includes — the detection, the tracking, the rule lookup, and the delivery. An unqualified number invites exactly one follow-up.

Q26.11 — How did you validate it with actual users?

Describe the supervisor feedback and what changed because of it.

If the honest answer is that it was a pilot with limited feedback, say that. "We had limited user feedback in the pilot window" is fine. Inventing user research is not.

Q26.12 — What are its privacy implications?

It's continuous video surveillance of identifiable people at work. That's significant and you should raise it yourself.

Mitigations: keep only the violation frames rather than all footage, restrict who can see them, set a retention limit, and tell workers the system exists.

Raising this before they do is one of the strongest maturity signals available to you.

Q26.13 — What would you build next?

Better detection on goggles and gloves specifically. Low-light robustness. Per-camera monitoring so you know when one degrades. And an evaluation set for the **rule-match-ing half**, which currently has no measurement at all.

Q26.14 — Why is there no accuracy number for the RAG half?

Because you didn't build an evaluation set for it. Own that directly.

Then describe the one you'd build: fifty violation-to-clause pairs, measuring how often the right clause comes back and whether the answer is actually supported by it. That's a day's work and you know exactly what it looks like.

Q26.15 — If I asked you to halve the cost of this system, what would you do?

Lower the frame rate. Drop to a smaller model and measure what accuracy you lose. Only run detection when motion is detected. Reduce the number precision. Batch across cameras. Use a smaller local language model.

Then say which of those costs accuracy and which is free. Frame rate and motion gating are nearly free; a smaller model is not.

DocuMind

Q27.1 — Describe DocuMind in 30 seconds.

"A document question-answering tool. You upload PDFs, it splits them up and indexes them, then you ask questions in plain English. Llama 3 runs locally through Ollama and answers in under two and a half seconds, showing you which passages it used."

Q27.2 — Why build it locally instead of using a hosted model?

Privacy and cost. The documents never leave the machine, and there's no per-question bill.

For confidential material that's the right trade, and privacy is the argument to lead with.

Q27.3 — What's the ingestion pipeline exactly?

Load the PDF, pull the text out, split it into 800-character chunks with 150 characters of overlap, turn each chunk into numbers, add them to the FAISS index, and save the index along with the chunk text and page references.

Q27.4 — What is the 2.5 seconds measuring?

Be precise. Is it the whole thing or just the search? Warm or cold? What size document? What hardware?

Say “end to end on my machine with the index already loaded.” Qualifying it is more credible than the bare number, and an unqualified latency figure invites exactly one follow-up.

Q27.5 — What dominates that latency?

Generating the answer, not finding the passages. Searching a few thousand vectors takes milliseconds; the model writing out the answer is nearly all of it.

Knowing where the time goes is the actual point of the question.

Q27.6 — How do you ensure answers are traceable?

Every chunk carries its source file and page. Those go into the prompt. The model is told to reference them. And the interface shows the retrieved passages so the user can check for themselves.

Q27.7 — What happens if the answer isn’t in the document?

The prompt tells it to say so rather than guess.

Test this explicitly — it’s the single most important behaviour in a document QA tool and the easiest one to get wrong. A tool that invents a plausible answer for a question the document doesn’t address is worse than useless.

Q27.8 — How would you handle a 500-page PDF?

In principle it already works, because you only retrieve a few passages regardless of document size.

What changes: ingestion takes longer, the index gets bigger, and chunking by section starts to beat chunking by character count. Mention batching the embedding step so it doesn’t take forever.

Q27.9 — What about tables, scanned pages and images?

Scanned pages need OCR first — there’s no text to extract. Tables come out scrambled without a layout-aware parser, because the columns get interleaved.

Be honest that this is a current limitation. Claiming it handles everything is one PDF away from being disproved.

Q27.10 — How would you support multiple documents and ask across them?

Store a document identifier with each chunk, let the user search within one document or across all of them, and cite which document each passage came from.

Also worth saying: this is where you’d outgrow FAISS, because it can’t filter by metadata natively.

Q27.11 — How do you know the answers are good?

Honest answer: there’s no formal evaluation yet.

Then describe one — thirty to fifty real questions with the expected source passage, measuring how often retrieval finds it and whether the answer is actually supported. Having the plan ready is most of the credit.

Q27.12 — What would you add next?

A reranker first — best quality improvement per hour of work. Then hybrid keyword search, conversational follow-ups, streaming output, and an evaluation harness.

Picking the reranker and saying why shows you can prioritise.

Fire detection

Q28.1 — Describe the fire detection project.

“A YOLOv5 single-class fire detector built with PyTorch and OpenCV. It runs on a webcam, a video file or still images. When it detects fire it emails an annotated frame as evidence — and the emailing happens on a separate thread so the video never stalls.”

Q28.2 — Why single-class rather than fire plus smoke?

Because that’s what the data supported — the config is one class, “fire”.

Adding smoke needs smoke annotations. And smoke is usually the **earlier** signal, so that’s the honest next step rather than a limitation you’re hiding.

Q28.3 — Why YOLOv5 here but YOLOv8 at Infosys?

A real answer is fine: v5 is what the project started on and had more examples available at the time.

Then note the actual architectural difference — v5 uses preset box shapes, v8 doesn’t — so it’s clear you know it isn’t just a version number.

Q28.4 — What’s your accuracy?

Don’t invent one. “I tested it on held-out data but I don’t have a number I’d stand behind on a resume, so I left it off.”

Then say what you **would** report: the detection score, and more importantly recall at your chosen threshold — because for fire, recall is the number that matters.

Q28.5 — What confidence threshold, and why?

Low. Recall dominates everything here. A missed fire is catastrophic; a false alarm costs one email.

Say plainly that you’d rather review false positives than miss an event.

Q28.6 — How do you suppress false alarms then?

Require the detection to appear across several consecutive frames before alerting.

It’s cheap, and it removes most single-frame flickers without raising the threshold — so you keep your recall.

Q28.7 — What triggers a false positive in practice?

Sunsets. Orange high-visibility clothing. Welding sparks. Headlights. Screens and monitors.

Fix by adding those as **negative examples** in training. This question is specifically checking whether you actually watched your model’s output or just read the metric.

Q28.8 — Explain the threading. What breaks without it?

Sending an email takes a couple of seconds. If that happens inside the loop that reads video frames, nothing reads frames during it — so the buffer fills and you drop frames or process stale ones.

A worker thread handles the email while OpenCV keeps reading. This is your best concrete engineering story in the whole project set: specific problem, you diagnosed it, clean fix.

Q28.9 — What if 100 detections fire at once?

Without a limit you'd spawn a hundred threads and spam the inbox.

The fix is a queue with one sender and a cooldown per event. Say whether you implemented it — identifying the flaw is worth more than pretending it isn't there.

Q28.10 — How are the SMTP credentials handled?

Environment variables, or a `.env` file excluded from Git. Never committed.

Check that's actually true of your repository before anyone clones it.

Q28.11 — How would you productionise this?

Multiple camera feeds. Confirmation across frames. Tiered alerts with acknowledgement and escalation if nobody responds. Edge hardware so it runs near the cameras.

And integration **alongside** certified smoke and heat sensors — never instead of them.

Q28.12 — Would you trust this as a building's fire alarm?

No, and say so plainly.

It's a supplementary early-warning layer. Life safety requires certified sensors with certification behind them. Claiming otherwise is the wrong answer in every room you'll ever be in.

Churn pipeline

Q29.1 — Describe the churn project.

"A production-shaped ML pipeline. A Random Forest on a 7,043-record telecom dataset, tracked in MLflow with automatic promotion of the best run, served through FastAPI, containerised with the UI via Docker Compose, built in GitHub Actions and deployed on Railway. The point of it was the pipeline, not the model."

That last sentence is important. It sets the frame before they find the accuracy number.

Q29.2 — What's the dataset?

The IBM telecom churn dataset from Kaggle — 7,043 rows, 1,869 churners, 5,174 who stayed.

Name it. Pretending a well-known public dataset is proprietary collapses immediately.

Q29.3 — What features matter most?

Tenure, contract type (month-to-month dominates), monthly charges and payment method.

Know your top features. Being unable to name them suggests you never looked at the model you trained.

Q29.4 — Why Random Forest?

A strong default on tabular data: no scaling needed, handles mixed data types, and gives you feature importance for free.

Then volunteer that gradient boosting would probably beat it, and that's what you'd try next. Offering the better alternative unprompted is a good signal.

Q29.5 — What accuracy, and what's wrong with that number?

76%. And get in front of it: always guessing “no churn” gets 73.5%, so accuracy was the wrong thing to measure. What matters is how many churners you caught.

Say it before they do. This single move is the highest-leverage thing in your whole interview.

Q29.6 — So is the model useless?

Not necessarily. It might rank customers usefully even if the thresholded accuracy is unimpressive — and ranking is what a retention team actually needs.

The honest answer is you can't tell, because you didn't log the ranking metrics. That's a process failure, not a mystery, and saying it that way is much better than defending the model.

Q29.7 — How would you fix it, concretely?

Log F1, ROC-AUC and PR-AUC. Set `class_weight='balanced'`. Tune the threshold against what a missed churner costs versus a retention offer. Try gradient boosting. Use stratified cross-validation instead of one split.

That's a couple of hours of work, and knowing it's a couple of hours is part of the answer.

Q29.8 — How does the MLflow promotion work, and what's its flaw?

It compares runs on the logged score and promotes the winner.

The flaw: the logged score is accuracy, so it can promote a model that's worse at the actual job. Naming your own design flaw is more impressive than hoping it goes unnoticed.

Q29.9 — Walk me through what happens when I POST to `/predict`.

Pydantic validates the incoming JSON. The preprocessing pipeline transforms it. The loaded model predicts. A probability and a label come back as JSON.

Be clear that the scaler and encoders are saved **inside** the pipeline file, so training and serving transform the data identically. That's your defence against train-serve skew.

Q29.10 — What does Docker Compose run here?

The FastAPI service and the Streamlit UI together, networked by service name, so `docker compose up` gives a reviewer the whole thing in one command.

Q29.11 — What does the CI pipeline actually check?

Import checks and the Docker image build. That's it.

Don't inflate it. And check it triggers on your real default branch, or it has never actually run.

Q29.12 — Your README says CI/CD is future work. Which is right?

The resume. The workflow file exists; the README is out of date. Fix it before you send the resume anywhere.

Q29.13 — Why Railway rather than AWS?

Zero cost and zero operations for a portfolio service.

Then show you know the trade: no autoscaling story, cold starts on first request, and with real traffic you'd move to a managed container service.

Q29.14 — What would you do differently if you rebuilt it?

Decide the metric **before** training. Log recall and PR-AUC. Use time-based splits. Add real tests and a minimum-score gate in CI. Monitor for drift.

In other words: decide what "good" means first. That's the lesson, and it's the right note to end on.

PRACTICE | The rehearsal that matters

Record yourself giving the SafeSight overview (Q26.1) and the churn overview (Q29.1). Both must be under sixty seconds and both must end somewhere you want them to dig. Then rehearse Q29.5 until raising the 73.5% baseline yourself feels completely natural.

CHAPTER RECAP

- Lead the churn description with "the point of it was the pipeline, not the model."
- Raise the 73.5% baseline before they find it.
- Name your own contribution to SafeSight precisely — the follow-up is always a detail.
- Raise the surveillance privacy question yourself; it's a strong maturity signal.
- Say plainly that the fire detector is not a fire alarm.
- "I didn't build an evaluation set, and here's the one I'd build" is a good answer.
- Every project has one weakness. Volunteer it, with the fix attached.

The Coding Round

Twenty things you should be able to write without an IDE

FOUNDATION

PREREQS: none

A screening round will ask you to write code in a shared document with no autocomplete. These are the patterns that come up. Practise writing them rather than recognising them — the gap between those two is much bigger than people expect.

WHAT YOU WILL LEARN

- State time and space complexity for anything you write
- Write the standard patterns from memory
- Explain your approach before you start typing
- Know the Python-specific answers that save you time

Complexity

Q30.1 — What are time and space complexity?

How much longer the code takes, and how much more memory it uses, as the input gets bigger. Written in big-O, which describes the growth rather than the exact number.

Always state both. Saying “this is $O(n)$ time and $O(1)$ space” unprompted scores points.

Q30.2 — Give the complexity of common operations.

Looking up a list by position: instant. Searching an unsorted list: you have to check everything. Searching a sorted list with binary search: halves each step, so very fast. Dictionary lookup: instant on average. Sorting: $n \log n$.

Q30.3 — Reverse a string.

`s[::-1]` . If they want the manual version, two pointers swapping inward from both ends.

Q30.4 — Check a palindrome.

`s == s[::-1]` . The manual version uses two pointers from each end moving inward, which uses no extra memory.

Q30.5 — Find duplicates in a list.

Walk through it keeping a set of what you've already seen. If something's already in the set, it's a duplicate.

One pass, and the set lookup is instant.

Q30.6 — Two-sum.

For each number, check whether `target - number` is already in a dictionary of what you've seen. If it is, you've found the pair. If not, record this number and move on.

One pass instead of checking every pair against every other pair.

Q30.7 — FizzBuzz.

Loop from 1 to n. Divisible by 15 print FizzBuzz, by 3 print Fizz, by 5 print Buzz, otherwise the number.

Check 15 first. Checking 3 first means you never reach the FizzBuzz case — that's the whole trick of the question.

Q30.8 — Fibonacci — iterative, recursive, memoised?

Plain recursion recomputes the same values enormously many times — don't offer it.

Iterative with two variables is the right answer: one pass, barely any memory. Memoised recursion caches results and is also fine.

Q30.9 — Reverse a linked list.

Three pointers — previous, current, next. Walk along, pointing each node backwards as you go.

One pass, no extra memory.

Q30.10 — Detect a cycle in a linked list.

Two pointers, one moving one step at a time and one moving two. If there's a loop, the fast one eventually laps the slow one and they meet. If there isn't, the fast one hits the end.

Q30.11 — Find the maximum subarray sum.

Walk through keeping a running total. Whenever the running total goes negative, reset it to zero — because a negative prefix can only hurt. Track the best total you've seen.

One pass.

Q30.12 — Sort a dictionary by value.

```
sorted(d.items(), key=lambda kv: kv[1], reverse=True).
```

Q30.13 — Count word frequencies in a string.

```
collections.Counter(text.lower().split()), then .most_common(n).
```

Q30.14 — Binary search — write it.

Two bounds, low and high. Look at the middle. If it's too small, move the low bound up past it. If too big, move the high bound down. Repeat until they cross.

Requires a sorted list — say that, because it’s half the point.

Q30.15 — Explain recursion and its risk in Python.

A function calling itself, working towards a base case where it stops.

The risk: Python caps how deep it’ll go at about a thousand levels. Go deeper and it crashes. So for anything with large input, use a loop.

Q30.16 — Stack vs queue?

A stack is last in, first out — like a pile of plates. A queue is first in, first out — like a queue of people.

In Python use `collections.deque` for both; it’s fast at either end, whereas a list is slow at the front.

Q30.17 — What is dynamic programming?

Breaking a problem into smaller versions of itself, and **saving the answers** so you never solve the same sub-problem twice.

That’s the whole idea. The Fibonacci example above is the simplest case.

Q30.18 — BFS vs DFS?

Breadth-first explores level by level using a queue — it finds the shortest path in an un-weighted graph.

Depth-first goes as deep as it can before backtracking, using a stack or recursion.

Q30.19 — Find the k largest elements.

`heapq.nlargest(k, arr)`.

If they want the reasoning: keep a heap of size k, and for each new element, if it’s bigger than the smallest in the heap, swap it in. That way you never sort the whole list.

Q30.20 — How would you process a 10 GB CSV on a laptop?

Read it in chunks rather than all at once, aggregating as you go. Only load the columns you actually need, and use smaller number types where you can.

Or use a tool built for it — Polars or DuckDB will handle it directly without you managing the chunking.

PRACTICE | How to behave in the round

Say your approach out loud **before** you type. “I’ll use a dictionary to record what I’ve seen, so it’s one pass instead of nested loops.” Then write it.

If you get stuck, say what you’re stuck on. An interviewer will almost always nudge you, and watching you reason is most of what they’re assessing.

CHAPTER RECAP

- State time and space complexity unprompted.
- Say your approach before you type it.
- Two pointers, a set of seen values, and a dictionary lookup cover a surprising share of these.
- Check 15 first in FizzBuzz.
- Binary search needs a sorted list — say so.
- Python's recursion depth is limited to about a thousand.
- For a huge CSV: chunks, fewer columns, or a different tool.

What to Ask Them, and the Final Checklist

The last twenty minutes, and the morning before

REFERENCE

PREREQS: none

Turning up with no questions reads as no interest, and it is the easiest thing in the whole interview to get right. Then the checklist — short, practical, and worth re-reading on the morning itself.

WHAT YOU WILL LEARN

- Ask three or four questions that tell you something real
- Know which questions reveal a weak team
- Walk in with your numbers memorised
- Fix the two things that would undermine you before anyone sees them

Questions to ask them

Ask three or four. Pick ones you actually want the answer to — it shows.

Q31.1 — What does the first 90 days look like in this role?

Tests whether they have a plan for you. Vagueness here is a genuine warning sign: it usually means nobody has thought about what you'll do.

Q31.2 — Is this role closer to building models or to deploying and maintaining them?

The single most useful question you can ask. The job title almost never tells you, and the daily work is completely different.

Q31.3 — What's already in production, and what's still at notebook stage?

Tells you whether you'll be building infrastructure from scratch or working inside something that already exists. Both are fine — but you want to know which.

Q31.4 — How do you measure whether a model is working once it's live?

A strong team has a concrete answer here. A weak one says “we check accuracy,” which tells you a great deal about how mature they actually are.

Q31.5 — Who labels your data, and how is label quality handled?

Nearly every real ML problem turns out to be a data problem. Their answer reveals whether they've encountered that yet.

Q31.6 — What does your path from a trained model to serving traffic look like?

Signals how much of the MLOps work you'd own versus inherit.

Q31.7 — How big is the team, and who would I be learning from?

You want to know whether there's anyone senior to review your work. As a junior this matters more than which framework they use.

Q31.8 — How are priorities decided between exploring and shipping?

Reveals whether the team gets room to try things or is purely delivery-driven.

Q31.9 — What's the biggest technical challenge the team is facing right now?

Gets you a genuine, unrehearsed answer — and gives you something concrete to respond to, which turns the end of an interview into a conversation.

Q31.10 — How does code review work here?

A team without review is a team where you won't improve. Worth knowing before you join rather than after.

Q31.11 — What would make someone unsuccessful in this role?

More revealing than asking what success looks like. People answer this one honestly, and you learn what actually goes wrong there.

Q31.12 — Is there support for continuing my M.Sc. alongside this role?

Ask it directly. Far better to surface a conflict now than six months in.

Q31.13 — What are the next steps, and when should I expect to hear back?

Always close with this. It sets a timeline and gives you a legitimate reason to follow up.

Before you walk in

Q32.1 — Fix the two stale READMEs.

The churn project lists CI/CD as future work when the workflow actually exists. The fire detection README describes an older design with email alerts as future work, when the code is YOLOv5 with threaded email already working.

If an interviewer reads either before talking to you, your resume looks inflated when it is in fact accurate. **This is the highest-value hour of preparation available to you.**

Q32.2 — Check your CI workflow's trigger branch.

If the file says it runs on `main` and your default branch is `master`, the pipeline has never run. Verify it before you claim it.

Q32.3 — *Know your numbers cold.*

0.841 and 0.467 for the two detection scores. Five safety-gear classes. 7,043 records, 26.5% churners, 73.5% baseline, 76% accuracy. 800-character chunks with 150 overlap. 384-dimension embeddings. 300 trees.

These should come out without hesitation. Hesitating on your own numbers suggests you didn't produce them.

Q32.4 — *Pre-empt the churn metric.*

Raise the 73.5% baseline yourself, before they ask.

It converts your weakest number into evidence of judgement. It is the single highest-leverage thing in this entire book.

Q32.5 — *Know which business percentages you can source.*

45% compliance, 25% audit efficiency, 25% visibility, 40% verification time.

For each one, decide now: measured by you, reported by the client, or an estimate. Then say which when asked. If you can't source one, take it off the resume rather than defend it live.

Q32.6 — *Test that each repo runs from clean.*

Clone into a fresh folder and follow your own README exactly. Interviewers do try this, and discovering a broken setup while they're watching is avoidable.

Q32.7 — *Have one story ready for each prompt.*

Failure, conflict, deadline, proudest achievement, hardest bug. One rehearsed story each, drawn from your four projects. Chapter 1 has the candidates.

Q32.8 — *Prepare three questions for them.*

Pick from the list above and adapt at least one to the specific company. A generic question is better than none; a specific one is much better than both.

Q32.9 — *Say "I don't know" when you don't.*

Then say how you'd find out.

Every experienced interviewer prefers that to a confident fabrication. And fabrication is the one failure they will not forgive, because it makes everything else you said unreliable.

Q32.10 — *Stop talking after the short answer.*

Answer in two or three sentences and let them follow up.

Over-explaining is how candidates walk into territory they can't defend. The silence after a short answer is not awkward — it's them deciding what to ask next, which means you're in control of the conversation.

KEY INSIGHT | The three things that matter most

If you only do three things from this entire book:

One. Fix the two READMEs and check the CI branch. One hour.

Two. Rehearse raising the 73.5% churn baseline yourself until it feels natural. Ten minutes.

Three. Record yourself giving the SafeSight and churn overviews, and cut both to under sixty seconds. Thirty minutes.

That is under two hours, and it addresses the three things most likely to decide the outcome.

CHAPTER RECAP

- Ask three or four questions, and ask the model-building-versus-deploying one.
- “How do you measure whether a model is working live?” tells you how mature they are.
- Fix the stale READMEs and check the CI trigger branch. One hour, highest value.
- Know your numbers without hesitating.
- Raise the 73.5% baseline yourself.
- “I don’t know, here’s how I’d find out” beats invention every time.
- Answer in two or three sentences, then stop.